

One to More, More to One: Category-Aware Iterative Expert Training for Software Engineering Agents

Jie Zhao^{*†} Ziyu Jiang^{*†} Suhang Zheng
Minghui Shan Xiaoxiao Xu Lin Qu

Alibaba Group

 **Model** · [Logics-SWE-Qwen3.6-27B](#)

Abstract

Repository-level software engineering (SWE) comprises heterogeneous task categories, whose progress under pooled agentic reinforcement learning can be uneven: gains in some categories coincide with regressions in others, while aggregate resolution obscures these changes. Motivated by this category see-saw, we develop a category-aware expert-training and policy-integration framework. Executable task construction and SWE Labeler, an evidence-grounded multi-axis labeling system, organize the training pools. Initial category-specific RL improves average training success while leaving uneven instance-level progress, motivating explicit consolidation of successful behavior and policy-adaptive task selection. Same-origin category experts alternate long-horizon Agentic-miniRL with Refresh-Repair-Expand (RRE): the updated policy refreshes instance mastery, reuses its own verified successful trajectories for Repair SFT, and reselects tasks for further RL. Label-routed multi-teacher on-policy distillation (MOPD) consolidates the experts into one deployable student, with ReLU-gated reward extrapolation keeping only each teacher’s improving direction over the reference. Expert training and policy integration require no external model to provide solution trajectories or action targets. We evaluate Pooled RL and Balanced RL, expert development, and single-model integration through aggregate and per-category resolution, the minimum category lift over each joint-RL baseline, and expert-gain recovery. The final MOPD policy achieves mean resolution of 58.04% on Pro-618 and 59.00% on SWE-bench Multilingual, improving over the base model by 5.39 and 2.78 percentage points, respectively.

1 Introduction

Software engineering agents must navigate repositories, edit code, invoke tools, inspect execution feedback, and revise a patch across long, stateful trajectories. Benchmarks and agent-computer interfaces have made this process executable and measurable [Jimenez et al., 2024, Yang et al., 2024, Wang et al., 2025d].

The breadth of software engineering itself makes this training problem structurally rich. Repository-level tasks span distinct engineering contexts: service and data-layer bug fixes, user-facing interface adjustments, infrastructure and tooling changes, and performance or security patches each demand different evidence sources, tool-interaction patterns, and verification approaches. Recent benchmarks

^{*}Equal contribution.

[†]Corresponding authors: aixiao.zj@alibaba-inc.com, jiangziyu.jzy@alibaba-inc.com.

make this heterogeneity concrete. SWE-bench Verified curates a human-validated subset of 500 Python instances with clear, solvable specifications [OpenAI, 2024]; SWE-bench Pro moves beyond the bug-fix-dominated setting of Verified to more varied repository contexts, maintenance intents, and modification scales [Deng et al., 2025]; and SWE-bench Multilingual extends the evaluation to 300 tasks across 42 repositories and nine programming languages [Khandpur et al., 2025]. Although all remain within repository-level SWE, they expose an agent to materially different task demands under a shared executable-verdict interface.

Existing post-training strategies can be grouped into three families, each leaving a distinct gap.

Joint training on the pooled mixture. Existing systems such as SWE-RL, SWE-Gym, and SWE-Master improve a shared SWE policy using pooled task or trajectory collections, with training recipes ranging from supervised fine-tuning to reinforcement learning [Wei et al., 2025, Pan et al., 2025, Song et al., 2026]. This treatment is convenient but suppresses structure that matters to optimization: a local logic fix, a cross-module compatibility change, a build repair, and a security patch can differ in trajectory length, reward variance, and data frequency. A joint update may therefore help one task category and hurt another even when both depend on overlapping underlying skills, yet a single aggregate metric cannot reveal this internal redistribution.

Pipeline decomposition into subtasks. A second family decomposes issue resolution into functional stages such as localization and repair. Agentless reduces the problem to hierarchical fault localization followed by repair [Xia et al., 2024]; SWE-Fixer trains dedicated retrieval and editing modules [Xie et al., 2025]; AutoCodeRover orchestrates structured subtask workflows [Zhang et al., 2024]. These approaches decompose the issue-resolution workflow into functional stages. Our decomposition is complementary: we partition the training task distribution into categories and iteratively develop category-specific experts before integrating them into a single policy.

Cross-domain expert splitting and fusion. A third line splits training by domain or capability and then consolidates experts. Branch-Train-Merge trains independent domain experts and merges them [Li et al., 2022, Sukhbaatar et al., 2024]; MOPD routes multiple teachers during on-policy distillation [Ma et al., 2026]; and ExOPD extrapolates beyond the teacher along a reference-anchored direction [Yang et al., 2026]. However, these methods operate across conventional domains such as mathematics, code generation, and instruction following, where the expert partition is given by domain boundaries. We investigate this specialization-and-integration approach within repository-level SWE, using observable task categories to organize expert training. Our motivation is that when aggregate scores are decomposed, opposing category-level changes can hide beneath stable overall metrics, a phenomenon we call the *category see-saw*.

This observation raises two questions: can balancing category exposure improve coordinated learning within a shared policy, and can category-specific training develop stronger specialists whose gains can be integrated into one model? Category separation alone does not guarantee stronger experts. In our initial expert RL runs, average training-instance success rates improve, but some instances regress while others improve. These paired observations motivate explicit consolidation of successful behavior and repeated reassessment of which tasks provide useful training signal. We use Refresh–Repair–Expand (RRE) to develop the category experts before evaluating their integration.

We take a single software engineering domain, organize its heterogeneous tasks into categories, and develop category experts through iterative Refresh–Repair–Expand training. We then bring these experts together through multi-teacher on-policy distillation, aiming to retain their capabilities in a single deployable agent (Figure 1). Cross-domain capability integration starts from domains that are separable by construction, so an expert partition is given; inside one conventional domain no such partition exists, and we instead establish category granularity through explicit rules over observable task labels. A hierarchical labeler makes these categories auditable, and each category line trains a same-origin expert by alternating stable long-horizon agentic RL with mastery refresh and Repair SFT. During label-routed multi-teacher on-policy distillation (MOPD), reward extrapolation augments imitation with a reference-anchored learning signal. Sparse environment reward defines executable correctness during expert training; routed teachers provide the dense policy signal during pure on-policy distillation.

Categories provide a common task-side structure for organizing training and measuring its outcomes. The central evaluation asks whether expert development and subsequent integration produce a single

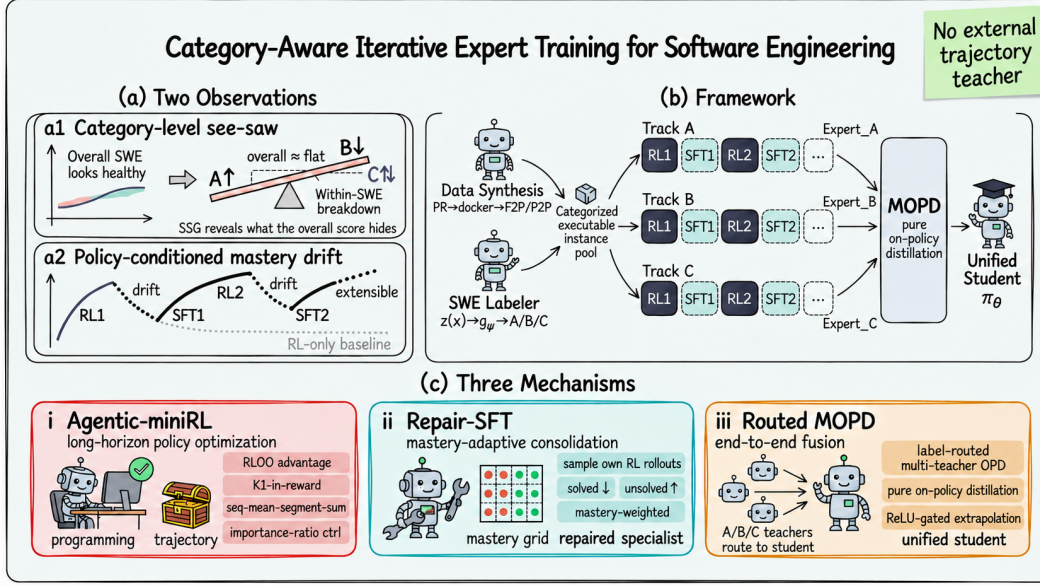


Figure 1: **Category-aware agentic RL for software engineering.** (a) Two observations motivate the framework: aggregate scores can conceal opposing category-level changes during joint RL, while a fixed task pool becomes stale as instance mastery drifts over long-horizon training. (b) SWE LABELER organizes executable tasks into categorized training pools; three same-origin training tracks alternate AGENTIC-MINI RL with Repair SFT before their experts are consolidated into one student through pure label-routed MOPD. (c) The three mechanisms provide long-horizon policy improvement, mastery-aware repair, and multi-teacher on-policy distillation without an external trajectory teacher.

model with stronger overall and per-category resolution than the Pooled RL and Balanced RL baselines.

Our contributions are:

- **Category-aware specialization and integration within SWE.** To our knowledge, this is the first study to organize repository-level SWE tasks into semantic repository-domain categories, train category-specific RL experts, and consolidate them into a single policy through label-routed multi-teacher on-policy distillation (MOPD). Category-level evaluation connects the motivating see-saw observations to the practical objective: improving overall and per-category resolution in one deployable policy.
- **Self-improving experts through Refresh–Repair–Expand.** Instance-level diagnostics reveal that average initial-RL gains coexist with observed regressions. RRE couples policy development with data selection: each updated expert refreshes instance mastery, reuses verified successes from its preceding RL trajectories for Repair SFT, and selects a new training frontier from the broader pool. AGENTIC-MINI RL supplies the long-horizon RL recipe. Expert development and policy integration use no external model to provide solution trajectories or action targets.
- **SWE Labeler: an evidence-grounded, multi-axis labeling system.** Source-grounded label definitions and explicit decision rules support both issue–change instances and agent interaction trajectories. Two hierarchical semantic axes and three ordinal scale axes provide a common foundation for category-level analysis, training-pool construction, and teacher routing. We apply the system to executable training data and multiple SWE benchmarks to characterize their task distributions.

2 Related work

Executable data and training environments for SWE agents. SWE-bench established repository-level issue resolution as an executable task, and SWE-Bench Pro extends this setting to broader, longer-horizon software changes [Jimenez et al., 2024, Deng et al., 2025]. Subsequent work has expanded the training substrate through collected repository tasks, procedurally generated instances, hybrid verifiers, and continuously refreshed evaluation: representative systems include SWE-Gym, R2E-Gym, SWE-smith, Skywork-SWE, and SWE-rebench V2 [Pan et al., 2025, Jain et al., 2025, Yang et al., 2025b, Zeng et al., 2025, Badertdinov et al., 2026]. The resulting executable feedback is scalable but not automatically reliable; recent audits identify specification, environment, grading, and reward-hacking failure modes, including incorrect patches that still receive positive verifier outcomes [Wang et al., 2026, Rajan, 2026, Zhao et al., 2026]. We use executable repository tasks as the training substrate for category experts, with task audits and runtime anti-hacking controls preserving the integrity of the outcome reward used by agentic RL.

Post-training software engineering agents. SWE-agent post-training has progressed beyond a single recipe. Trajectory supervision trains repository navigation, localization, and editing behavior, as illustrated by SWE-Fixer and SWE-Dev [Xie et al., 2025, Wang et al., 2025a]. Recent SFT-centered systems further scale validated trajectories, difficulty curricula, and execution-backed refinement: SWE-Lego studies a strong SFT-only recipe, whereas SWE-ZERO to SWE-HERO moves from execution-free supervision to a smaller execution-validated stage [Tao et al., 2026, Ludwig et al., 2026]. SWE-RL studies reinforcement learning with patch-similarity rewards [Wei et al., 2025], while long-context multi-turn RL uses execution feedback for repository-level tasks [Golubev et al., 2025]. SWE-Master and SWE-Protégé combine data curation, supervised fine-tuning, reinforcement learning, or selective expert assistance in broader post-training pipelines [Song et al., 2026, Kon et al., 2026]. Rather than treating SWE as one homogeneous post-training distribution, we work at a finer granularity within this broad domain: evidence-grounded categories define separate expert-training streams, and each expert is progressively refined by agentic RL and Repair SFT drawn from its own successful rollouts.

Reinforcement learning for long-horizon agents. Multi-turn agents interleave language-model actions with stateful tool and environment observations, yet often receive only a sparse task-level outcome. This regime has motivated algorithms and empirical studies beyond single-turn RLVR. DAPO develops large-scale policy-optimization practices, ARPO targets credit attribution and exploration around tool interactions, and recent multi-turn studies analyze how environment complexity, reward sparsity, policy gradient estimators, and horizon length affect learning stability [Yu et al., 2025, Dong et al., 2025, Wang and Ammanabrolu, 2025, Kim et al., 2026]. For repository-level SWE, long-context multi-turn RL demonstrates the importance of optimizing directly against environment feedback [Golubev et al., 2025]. MiniRL further relates stable LLM RL to train–inference discrepancy and policy staleness [Zheng et al., 2025]. Agentic-miniRL instantiates these principles for long-horizon SWE trajectories through behavior-policy correction, RLOO advantages, the K1 reference penalty in the reward path, and turn-aware loss reduction over assistant actions separated by tool observations.

Heterogeneous SWE tasks and category-aware specialization. Repository-level SWE contains materially different problem structures even when all instances share the same agent interface and executable success criterion. Recent benchmarks make this variation concrete: visual software issues require multimodal grounding and interface-state reasoning, security tasks require vulnerability reproduction and patch validation, and performance engineering requires bottleneck localization and optimization under workload constraints [Yang et al., 2025a, Lee et al., 2025, Shetty et al., 2025]. Related post-training studies show that gains learned in one domain need not transfer uniformly to others, and that sample interactions or optimization conflicts can shape multi-domain outcomes [Hu et al., 2026, Liang et al., 2025b, 2026, Ming et al., 2026]. These works mainly study differences among broad domains or capabilities. We instead examine heterogeneity *within* SWE: a hierarchical labeling system maps observable task evidence to fine-grained annotations, from which we derive a small number of trainable categories. This structure exposes category see-saw hidden by aggregate scores and provides the routing basis for training and evaluating category-specific experts.

Model merging and multi-teacher on-policy distillation. Expert models can be integrated either in parameter space or through teacher supervision. Task arithmetic and branch–train–merge combine independently specialized parameter updates without teacher inference [Ilharco et al., 2022, Li et al., 2022]. Knowledge distillation instead transfers output distributions; GKD and MiniLLM evaluate teachers on student-generated sequences, reducing the distribution mismatch of static teacher trajectories, while on-policy context distillation conditions a teacher on additional context [Agarwal et al., 2023, Gu et al., 2023, Ye et al., 2026]. MOPD extends on-policy distillation to routed domain teachers, and ExOPD generalizes the objective with reward extrapolation; recent analysis also identifies optimization-budget imbalance as a source of incomplete multi-teacher integration [Ma et al., 2026, Yang et al., 2026, ang Gao et al., 2026]. Our setting couples these ideas to intra-domain specialization: all experts originate from the same SWE base model, their granularity and routing are defined by observable category labels, and their behaviors are integrated on the student’s own long-horizon trajectories using label-routed MOPD with reward extrapolation.

3 Problem setup and empirical motivation

We first formalize executable repository-level SWE tasks and the long-horizon agents that interact with them. We then show how aggregate joint-RL performance can conceal opposing movements across heterogeneous SWE categories. Finally, a structured multi-axis SWE label space, mapped into operational categories by deterministic rules, makes this heterogeneity measurable and provides the basis for category-aware specialization and integration.

3.1 Repository-level SWE and joint agentic RL

Following executable issue-resolution benchmarks such as SWE-bench, we model a software-engineering instance as

$$x = (\mathcal{R}, b, q, \mathcal{E}, \mathcal{V}). \quad (1)$$

Here $\mathcal{R}$ is a repository, b is the base revision at which the issue is reproduced, q is a natural-language problem statement, $\mathcal{E}$ is a reproducible execution environment, and $\mathcal{V}$ is an executable verifier [Jimenez et al., 2024]. The environment specifies the source tree, dependencies, runtime, and permitted tools. The verifier contains task-specific tests and non-regression checks. The evaluated agent observes q , the repository at b , and feedback from tools it invokes; it does not observe the reference patch or hidden verifier outcomes.

Starting from $\mathcal{R}_0 = \text{checkout}(\mathcal{R}, b)$, an agent produces a terminal repository state $\mathcal{R}_T$ and hence a patch $p = \text{diff}(\mathcal{R}_0, \mathcal{R}_T)$. In the binary setting used throughout this paper, executable success is

$$r^{\text{env}}(x, p) = \mathbb{I}[\text{apply}(p) \wedge \mathcal{V}_{\text{F2P}}(\mathcal{R}_T) \wedge \mathcal{V}_{\text{P2P}}(\mathcal{R}_T)], \quad (2)$$

where fail-to-pass tests encode the requested repair and pass-to-pass tests guard against regressions. Environment construction failures, timeouts, and invalid patches are retained as separate audit outcomes rather than silently interpreted as evidence about a task category. This definition distinguishes repository-level SWE from isolated code generation: success depends on a stateful edit that remains compatible with the surrounding project.

Long-horizon tool-using policies. A SWE agent couples a language-model policy with an agent–computer interface that exposes repository navigation, file inspection and editing, shell execution, and test feedback [Yang et al., 2024, Wang et al., 2025d]. Let s_t denote the environment state, including the current repository and process state. The interface reveals an observation $o_t = \Omega(s_t)$ and executes a structured action $a_t \in \mathcal{A}$, inducing

$$s_{t+1} \sim P(\cdot \mid s_t, a_t), \quad \tau = (o_1, a_1, o_2, a_2, \dots, o_T, a_T). \quad (3)$$

Actions include search and inspection, patch-producing edits, commands or tests, and a terminal submission. Tool output, compiler or test errors, and the accumulated repository changes become subsequent observations. Because the verifier is partly hidden and the complete repository state is not placed in every model context, the policy acts on the interaction history $h_t = (q, o_{1:t}, a_{1:t-1})$:

$$a_t \sim \pi_\theta(\cdot \mid h_t), \quad J_{\text{RL}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \tau \sim \pi_\theta} [r^{\text{env}}(x, p(\tau))]. \quad (4)$$

The scalar objective is appropriate for executable correctness, but it aggregates over tasks with different maintenance intents and repository contexts. Consequently, two checkpoints with similar J_{RL} can have materially different distributions of solved tasks.

3.2 Empirical motivation: the category see-saw

The Pooled RL trajectories motivating this study show uneven category-level progress: improvements on some task groups coincide with regressions on others, while aggregate resolution obscures these differences. We call this observed pattern the *category see-saw*.

The underlying concern is broader than any one benchmark partition. Repository-level SWE is not a homogeneous problem distribution. Recent benchmarks make this heterogeneity concrete: visual JavaScript issues add multimodal grounding, asynchronous programming, and DOM/state manipulation [Yang et al., 2025a]; security engineering includes vulnerability reproduction, proof-of-concept generation, and patching [Lee et al., 2025]; performance optimization centers on bottleneck localization and low-level code [Shetty et al., 2025]; and research-repository tasks emphasize environment setup, configuration, and execution [Bogin et al., 2024]. Although all remain within repository-level SWE, these task families expose an agent to different evidence sources, interaction patterns, and executable success criteria. Joint RL nevertheless updates one shared policy from a pooled stream under finite sampling and update budgets. Learning can therefore progress unevenly across these task-demand mixtures: experience that benefits one category may transfer only partially to others. Closely related trade-offs have been observed in multi-domain LLM fine-tuning and multi-domain RL, where sample interactions evolve throughout training and gains in one domain can come at the expense of another [Liang et al., 2025b, 2026]. The categories used here are observable operational groupings of tasks; they do not identify isolated latent capabilities or a particular conflicting gradient.

We study this within-SWE heterogeneity on Pro-618, a fixed, audit-filtered subset of SWE-bench Pro. The repository-domain label system in Section 4.2 is defined over general SWE instances. Applying its A/B/C grouping to Pro-618 yields three mutually exclusive evaluation groups: service/data/security (*Pro-A*), user-facing applications (*Pro-B*), and systems, tooling, and runtimes (*Pro-C*), containing 221/201/196 tasks, respectively. We evaluate on 618 of the 731 SWE-bench Pro instances. Community reports have repeatedly flagged individual SWE-bench Pro tasks for faulty environments, broken container images, or unreliable evaluation logic. Rather than quietly editing the evaluation set to improve our scores, we exclude these instances through an explicit filter grounded in those public findings. Section 5.4 explains the benchmark choice and audit filter; Appendix A records the construction protocol.

Let $S_c(t)$ be the success rate of checkpoint t on category c , with $t = 0$ denoting the base model, and let $\Delta_c(t) = S_c(t) - S_c(0)$. The corresponding overall gain is $\Delta_{\text{all}}(t) = \sum_{c=1}^K w_c \Delta_c(t)$, where w_c is the category share used by the official micro average. We quantify simultaneous progress across categories by

$$G_{\text{sim}}(t) = \min_c \Delta_c(t), \quad G_{\text{sim}}^* = \max_{t \in \mathcal{T}} G_{\text{sim}}(t), \quad (5)$$

where $G_{\text{sim}}(t) > 0$ means that every category improves over the base model at checkpoint t , and G_{sim}^* is the largest gain retained jointly by a single checkpoint in the evaluated trajectory $\mathcal{T}$. To measure how far the weakest category lags behind the overall result, we define the *see-saw gap* (SSG):

$$\text{SSG}(t) = \Delta_{\text{all}}(t) - G_{\text{sim}}(t). \quad (6)$$

The SSG is zero when all category gains are equal. It grows when aggregate improvement is concentrated in only part of the benchmark or when the overall score hides a category regression. At one checkpoint, SSG measures how far the least-improved category trails the aggregate gain; G_{sim} measures how much improvement is shared by every category. Across checkpoints, opposed category transitions characterize the temporal see-saw. These summaries satisfy $\Delta_{\text{all}} = G_{\text{sim}} + \text{SSG}$. Table 1 interprets representative changes relative to our objective. We seek higher overall and minimum category gains, with a narrower gap indicating better alignment between them. Gap reduction through aggregate decline alone does not meet this objective; category-wise score comparisons remain necessary to establish improvement in every category.

Figure 2 shows that under Pooled RL, gains in some categories coincide with regressions in others (panel a). Positive overall gains can coexist with a negative minimum category gain (panel b), showing that aggregate improvement alone does not guarantee progress in every category.

These observations motivate two approaches to category-aware post-training. The first adjusts the data composition of a shared training stream. Training mixture proportions affect multi-capability performance [Ming et al., 2026], making category balancing a natural intervention to evaluate. We instantiate this approach with a subset that matches category counts and base-model pass-rate strata.

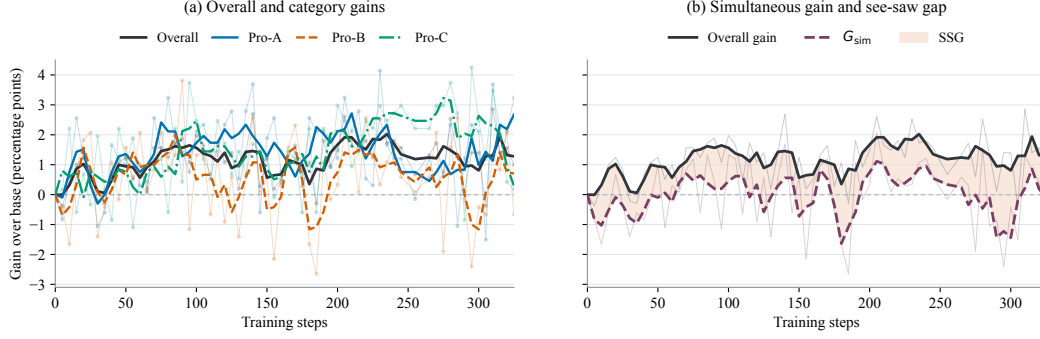


Figure 2: **Category dynamics under Pooled RL.** A single policy is trained on the full 6,723-task mixture and evaluated on the fixed Pro-618 split (Pro-A/B/C: 221/201/196 tasks). Faint traces show the mean at each checkpoint; bold traces show centered three-checkpoint moving averages. Step 0 anchors the base model. Left: overall and category gains relative to their respective base-model rates. Right: overall gain and minimum category gain G_{sim} ; their shaded difference between the smoothed curves is the smoothed SSG.

Table 1: **Interpreting representative changes in overall and category gains.** G_{sim} is the minimum category gain over the base; SSG is the gap between overall and minimum category gains. Arrows compare two checkpoints or training settings using the same base and category weights.

Full gain	G_{sim}	SSG	Interpretation relative to our objective
↑	↑	↓	Aligned improvement: overall and minimum gains rise while the gap narrows.
↑	↑	→	Shared improvement with a stable gap: overall and minimum gains rise together, with unchanged separation.
↑	↑	↑	Shared improvement with a wider gap: both gains rise, but overall gain increases faster than minimum gain.
↑	→ / ↓	↑	Aggregate improvement without minimum-gain improvement: overall gain rises while minimum gain stays unchanged or declines.
↓	→ / ↓	↓	Gap reduction through aggregate decline: the gap narrows without improving the minimum gain; this does not meet our improvement objective.
↓	↑	↓	Minimum-aggregate trade-off: the minimum gain improves and the gap narrows at the cost of overall gain.

Full gain = $G_{\text{sim}} + \text{SSG}$; → denotes no change. Improvement in every category must still be checked against the A/B/C scores.

The second changes the organization of policy learning: train experts on category-specific pools, then integrate their behaviors into a single model. In particular, does category-specific RL alone produce experts that outperform Pooled RL on their respective categories? Our experiments first examine Balanced RL, then assess the initial experts and their subsequent development before evaluating single-model integration.

3.3 From category-aware diagnosis to specialization and integration

The see-saw above exposes a limitation of aggregate evaluation: a single resolution score cannot show where joint RL improves, stagnates, or regresses within the heterogeneous SWE task distribution. We therefore use a structured label space that is expressive enough to describe diverse SWE instances and organized enough to support consistent category-level aggregation. It provides a common unit for measuring category-wise progress, constructing balanced controls, and forming specialized training pools. We write

$$\mathbf{z}(x) = L_\phi(e(x)), \quad c(x) = g_\psi(\mathbf{z}(x)) \in \mathcal{C} \cup \{\emptyset\}, \quad (7)$$

where $e(x)$ is offline task evidence, L_ϕ is a frozen labeler, g_ψ is a deterministic category rule, and $\emptyset$ denotes an unrouted task. The fine-grained vector $\mathbf{z}(x)$ retains instance properties, whereas g_ψ maps them into operational categories. At checkpoint t , this grouping expands the scalar overall result

into the category-performance vector $(S_1(t), \dots, S_K(t))$, revealing how gains and regressions are distributed across the SWE task space. The same map is applied to training-pool construction, expert routing, and category-aware evaluation.

SWE LABELER, described in Section 4.2, instantiates this structured label space. Its multiple axes describe observable instance properties such as repository domain, maintenance intent, and modification scope. The main experiments derive Pro-A/B/C from a frozen Domain L1 rule; the remaining axes support within-category stratification and diagnosis. Here *repository domain* denotes the software ecosystem represented by a repository, while all labels remain within the overarching domain of repository-level SWE. Pro-A/B/C are therefore one coarse operational grouping of a richer annotation space. The same label space can organize training and held-out tasks independently of whether they originate from Pro-618.

A *label* is an observable instance annotation, a *category* is an operational grouping of labels, and a model *capability* is latent. One category can require several capabilities, and one capability can support several categories. Our analysis therefore concerns category-wise performance, specialization, and integration rather than a decomposition of latent capabilities.

Our learning problem is therefore to produce a single deployable policy with stronger aggregate resolution and category-level performance than Pooled RL and Balanced RL. We evaluate the overall score and score differences across all evaluation categories. The SSG separately summarizes how unevenly those gains are distributed. Balancing adjusts category exposure within one policy’s training stream. Our expert-based approach instead allocates a separate training stream to each category and then uses label-routed multi-teacher on-policy distillation (**MOPD**) to consolidate the experts’ behaviors into one deployed policy. The categories control training-pool construction and teacher routing, while the deployed agent receives only ordinary task and tool observations. Section 4 develops this label-grounded training framework.

4 Method

Our framework organizes SWE post-training around category experts and their integration into one deployed policy. Executable task construction provides the instances and verifiers, while SWE Labeler supplies the task categories. Each category expert starts from a common base and follows the Refresh–Repair–Expand (RRE) loop, using Agentic-miniRL for its RL phases and its own verified trajectories for Repair SFT. Label-routed multi-teacher on-policy distillation then consolidates the experts into a single student.

4.1 Executable SWE task construction

We construct the executable task pool from merged repository PRs and linked issues. Each instance contains a base revision, an issue-style problem statement, an environment specification, and executable tests. Fix patches are separated from test patches, and an LLM rewrites the PR and issue content into the problem statement. Leakage checks reject statements exposing source or test paths or added patch code, with boilerplate exempted; deduplication and contamination audits further filter the pool. Environment construction follows RepoLaunch [Zhang et al., 2025], producing a Docker image with rebuild and test commands.

We validate each instance by running its tests before and after the gold patch, retaining fail-to-pass and pass-to-pass tests. Repeated gold-patch checks filter flaky or time-decayed verifiers. Retained instances record provenance, verifier status, labeler version, and split membership. The resulting pool contains approximately 32K executable candidates; Section 5 describes the subsets used for training and RRE expansion. These tasks provide the executable training substrate, while the labeling system below supplies the categories used for expert training and teacher routing.

4.2 SWE Labeler: an evidence-grounded hierarchical labeling system

Repository-level SWE instances differ in maintenance intent, software ecosystem, and task scale. To make this heterogeneity actionable, we introduce SWE LABELER, an evidence-grounded hierarchical labeling system. Crucially, the taxonomy is constructed from established software-engineering concepts rather than a flat, benchmark-specific inventory: each fine-grained label links an authoritative

source to an operational SWE rule, observable evidence signals, and explicit boundaries with neighboring labels. The resulting label space is interpretable, auditable, and reusable for data construction, expert routing, and category-aware evaluation.

Label space. Each instance receives two semantic axes—*Task Type* and *Repository Domain*—and three orthogonal scale axes: modification scope, cognitive complexity, and estimated resolution time. Task Type captures the maintenance intent of a change; Repository Domain captures the software ecosystem of the repository; and the scale axes characterize the extent of modification, the reasoning complexity, and the anticipated resolution time. Given an instance x , the Labeler produces the semantic and scale annotation

$$\mathbf{z}(x) = (z_{L1}^{\text{task}}, z_{L2}^{\text{task}}, z_{L1}^{\text{domain}}, z_{L2}^{\text{domain}}, z^{\text{scope}}, z^{\text{cognitive}}, z^{\text{time}}). \quad (8)$$

The final taxonomy contains 26 Task Type L1 families with 119 L2 labels and 21 Repository Domain L1 families with 108 L2 labels. Each semantic axis has one primary L1 label; its L2 field retains one to three applicable children, or *unspecified* when no concrete child is supported by the evidence. The three scale axes each have four ordered levels, for 12 scale levels in total. The Labeler additionally records programming language, specification type, and dependency context as auxiliary metadata.

The full inventory and representative source chains appear in Appendix B.2; Figure 3 visualizes the breadth of the two semantic axes.

Taxonomy grounding and label contract. The taxonomy operationalizes established software-engineering concepts. The Task Type axis draws on the maintenance and quality vocabulary of ISO/IEC 25010 and ISO/IEC/IEEE 14764, defect and security mechanisms from MITRE CWE, and behavior-preserving restructuring from Fowler’s refactoring catalog [International Organization for Standardization, 2023, 2022, MITRE Corporation, 2026a,b, Fowler, 2018]. The Repository Domain axis organizes the software ecosystem in which a change occurs; its fine-grained definitions are grounded in primary framework, platform, and project documentation. The three scale axes combine repository topology with empirical SWE task analysis and cognitive-load theory into four-level operational definitions [Brand and Denain, 2025, Sweller, 1988].

At the fine-grained layer, every concrete label is a decision record

$$\mathcal{E}(\ell) = (a_\ell, d_\ell^{\text{src}}, d_\ell^{\text{op}}, s_\ell^+, b_\ell), \quad (9)$$

where a_ℓ identifies an authoritative source, d_ℓ^{src} its source concept, d_ℓ^{op} the operational SWE rule, s_ℓ^+ observable positive signals, and b_ℓ boundaries to neighboring labels. The current inventory instantiates this contract for all 119 Task Type L2 labels, 108 Repository Domain L2 labels, and 12 scale levels. For example, `bug-fix.logic_error` is grounded in CWE-682 and CWE-697: it covers an incorrect condition, calculation, or comparison, while `index-boundary mistakes` map to `off_by_one` and `unintended mutation maps` to `state_corruption`. This source-to-rule-to-boundary structure gives the Labeler explicit semantics at the point where neighboring classes are most easily confused. Appendix B details the label inventory, annotation procedure, structural audit, and category mapping.

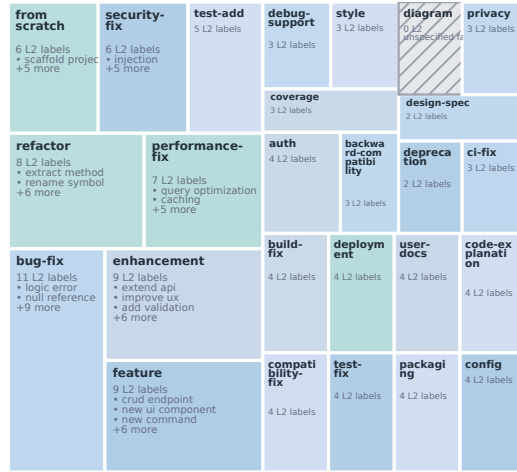
Task and trajectory evidence. Repository-level SWE is observed through two complementary evidence forms. Benchmark and mined instances are typically static issue–change tasks (e.g., issue–PR pairs), pairing a task statement and repository state with a reference change and executable tests. Interactive settings instead yield complete user–agent trajectories, which preserve the user request together with agent actions, tool observations, edits, and verification outcomes. Both forms describe SWE work, but they expose different evidence for the same label decision. Two input adapters normalize them into a common instance representation, after which the Labeler applies the same taxonomy and decision rules.

For GitHub issue–change instances, the annotation input includes the repository name and available GitHub metadata (description, topics, and primary language), the problem statement, and bounded excerpts of the code patch and test patch. Repository metadata establishes the project’s purpose and software ecosystem; the task statement and changes provide evidence of maintenance intent, affected functionality, and change scale. Both annotation stages receive this evidence. If repository metadata is unavailable, the Labeler falls back to the repository name and task/change content and marks this fallback in its domain rationale.

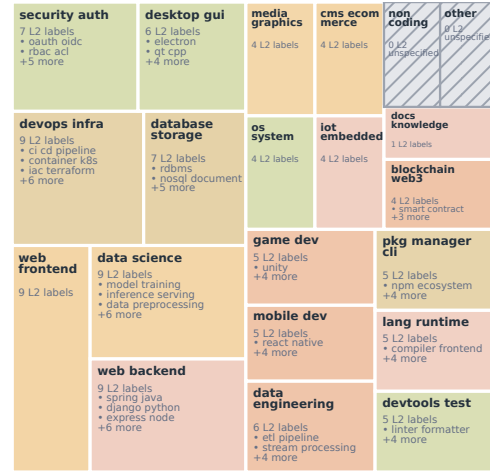
A rich, factorized taxonomy for software-engineering tasks

47 semantic families • 227 fine-grained labels • 3 orthogonal axes / 12 scale levels

(a) Task Type — 26 L1 / 119 L2



(b) Repository Domain — 21 L1 / 108 L2



Tile area uses a 3-unit visibility floor, then follows L2 count; hatched gray tiles have no concrete child. Structural inventory, not sample frequency.

Figure 3: **Semantic breadth of the SWE Labeler taxonomy.** The two panels show the 26 Task Type and 21 Repository Domain L1 families. Tile area is proportional to the number of declared L2 labels, and tile text shows selected L2 examples. The inventory represents the semantic resolution available to the Labeler.

Table 2: **Cross-benchmark profiles produced by SWE Labeler.** The three labeled corpora contain 1,531 instances. Percentages are computed within each corpus; the last column reports a representative high-mass scale label.

Labeling corpus	N	Langs.	Leading Task Type	Leading Repository Domain	Scale signature
SWE-bench Verified	500	1	<i>bug-fix</i> (87.0%)	<i>web_backend</i> (46.4%)	86.2% single-file
SWE-bench Pro	731	3	<i>bug-fix</i> (47.3%)	<i>devops_infra</i> (33.2%)	59.2% cross-module
SWE-bench Multilingual	300	8	<i>bug-fix</i> (89.3%)	<i>lang_runtime</i> (24.7%)	66.0% single-file

Annotation proceeds hierarchically. In the first stage, the Labeler predicts Task Type L1 and Repository Domain L1 from the broad record, together with the auxiliary metadata. Repository purpose and metadata provide the primary signal for the Domain label, while change-specific evidence resolves multi-purpose repositories. In the second stage, the Labeler selects L2 labels only from the children of the predicted L1 families and assigns the three scale axes. This conditional refinement concentrates the fine-grained decision on semantically nearby alternatives; the output is checked against the declared L1–L2 hierarchy and admissible scale levels. The trajectory adapter and the issue–change adapter thus produce comparable labels across task records and interaction trajectories.

Cross-benchmark label profiles. We applied the completed Labeler to 1,531 instances drawn from three SWE benchmark corpora. The collection spans SWE-bench Verified, SWE-bench Pro, and SWE-bench Multilingual, covering varied maintenance tasks and mirroring the recent expansion of executable SWE data across repositories and language ecosystems [Deng et al., 2025, Badertdinov et al., 2026]. Table 2 shows that the same label space captures sharply different maintenance, ecosystem, and scale profiles.

Routable category construction. The complete annotation $\mathbf{z}(x)$ is deliberately richer than a direct expert router: assigning one expert to every Repository Domain L1/L2 label would fragment the available RL examples and optimization budget. We therefore use Repository Domain L1 as the routing axis. It identifies the software ecosystem in which an agent must locate evidence, interact with tools, and validate a change; Task Type may occur across these ecosystems, while Task Type, L2 labels, and scale axes remain available for within-category stratification and fine-grained analysis.

For the main configuration, we coarsen the 18 routable Domain L1 families into three broad engineering contexts: service/data/security (A), user-facing applications (B), and systems, tooling, and runtimes (C). This three-route configuration instantiates category-aware specialization and integration under a fixed RL budget while retaining the full label vector for analysis. We construct the routes with a deterministic mapping from Repository Domain L1:

$$c(x) = g_\psi(z_{\text{L1}}^{\text{domain}}(x)) \in \{A, B, C, \emptyset\}. \quad (10)$$

The complete Domain L1 whitelist is given in Appendix B.4. The same rule is applied to all labeled training and evaluation records. Applied to the Pro-618 longitudinal probe, its three routes are denoted Pro-A/B/C. The resulting partition controls joint-RL sampling, category-expert training, and **MOPD**; the same labeled record also supports category-aware evaluation.

4.3 Agentic-miniRL

We build on the policy-gradient variant of MiniRL [Zheng et al., 2025] and instantiate it for long-horizon, multi-turn SWE trajectories. Four optimization challenges determine the objective. First, executable verification normally provides a sparse binary terminal reward. Group-relative learning obtains a nonzero executable-reward signal only from mixed-outcome rollout groups, which are scarce for difficult instances; we use RLOO to avoid further attenuating these signals through a self-inclusive baseline. Second, repeated long-horizon updates can move the policy away from a fixed reference; we regularize this drift through K1 in the token-reward path. Third, the rollout behavior policy μ_b , the pre-update learner $\pi_{\theta-}$, and the current learner π_θ need not coincide. We therefore correct for behavior-probability mismatch and bound policy updates at the token level. Fourth, a trajectory interleaves assistant action turns with tool observations, and the action turns vary widely in length; we aggregate the token loss turn by turn.

For each instance, μ_b produces G trajectories and their executable returns. The trainer stores the behavior log-probabilities, recomputes log-probabilities under $\pi_{\theta-}$, and evaluates a reference π_{ref} that remains frozen within the RL phase. MiniRL supplies the behavior-corrected token-level policy surrogate. We combine this surrogate with instance-wise RLOO advantage estimation, K1 reward regularization, and turn-aware reduction for Agentic SWE. The resulting computation follows the four operations above in order; only the final policy loss is differentiated.

RLOO reward centering: preserving finite-group signal. Repository tasks differ substantially in their probability of being solved, while executable verification provides only sparse terminal feedback. Comparing rollouts of the same instance conditions the learning signal on the task, but a learned critic for long, sparse-reward trajectories introduces an additional estimation problem. We instead adopt the leave-one-out (RLOO) [Ahmadian et al., 2024] baseline. For G rollouts of the same instance with executable returns $R_1, \dots, R_G$, RLOO excludes each rollout from its own baseline:

$$\bar{R}_i = R_i - \frac{1}{G-1} \sum_{j \neq i} R_j = \frac{G}{G-1} \left(R_i - \frac{1}{G} \sum_{j=1}^G R_j \right). \quad (11)$$

Under i.i.d. on-policy sampling, the leave-one-out baseline for trajectory i is independent of that trajectory conditional on the SWE instance. Its score-function contribution therefore has zero expectation, $\mathbb{E}[b_{-i} \nabla_\theta \log \pi_\theta(\tau_i | x)] = 0$, so subtracting b_{-i} preserves the expected REINFORCE policy gradient while providing a parameter-free, instance-conditioned control variate for variance reduction. This property is particularly useful for Agentic SWE: executable feedback arrives only after a long tool-interaction trajectory, value estimation would span heterogeneous intermediate tool states, and each sandbox rollout is expensive. RLOO reuses every trajectory both as an update sample and as baseline evidence for the other group members, without training a critic.

Like mean-centered GRPO, RLOO supplies a within-instance learning signal; its specific distinction is excluding the current rollout from its own baseline, whose finite-group effect we quantify below.

For binary executable rewards, suppose that k of the G rollouts succeed. Equation 11 assigns

$$\bar{R}^+ = \frac{G-k}{G-1}, \quad \bar{R}^- = -\frac{k}{G-1} \quad (12)$$

to every successful and failed rollout, respectively. In a non-degenerate group containing both successful and failed rollouts, a rare successful trajectory therefore receives a strong positive weight, while its failed siblings provide instance-matched negative evidence. An all-failure or all-success group has no within-instance contrast, which motivates pairing group-relative learning with the pass-rate-aware instance selection described in Section 4.4.

The rightmost form of Equation 11 also makes the relation to our mean-centered, non-standardized GRPO baseline explicit. If $A_i^{\text{GRPO}} = R_i - G^{-1} \sum_j R_j$, then

$$\bar{R}_i = \frac{G}{G-1} A_i^{\text{GRPO}}. \quad (13)$$

For the centered executable reward, mixed-outcome groups are the only groups with nonzero advantages. Including a rollout in its own group mean would shrink every such advantage by $(G-1)/G$. RLOO restores this finite-group scale without changing the sign or relative ordering of the advantages. In our $G=8$ configuration, for example, the sole successful rollout in a one-success group receives advantage 1 under RLOO, compared with $7/8$ under inclusive mean centering. The emphasis on scarce successful behavior still comes from within-instance comparison; the purpose of RLOO is to avoid further attenuating that already limited signal. Equations 11–13 match the centering operation used in our implementation.

K1 regularization before return-to-go. Long-horizon training can accumulate drift from a fixed reference policy. We therefore place the sampled K1 log-ratio in the reward path, after RLOO has centered the executable return and before return-to-go is computed. For each valid assistant token,

$$\begin{aligned} d_{i,t}^{\text{K1}} &= \log \pi_{\theta^-}(a_{i,t} \mid h_{i,t}) - \log \pi_{\text{ref}}(a_{i,t} \mid h_{i,t}), \\ \tilde{r}_{i,t} &= \mathbf{1}[t = T_i] \bar{R}_i - \beta d_{i,t}^{\text{K1}}, \quad A_{i,t} = \sum_{u=t}^{T_i} \gamma^{u-t} \tilde{r}_{i,u}, \end{aligned} \quad (14)$$

where T_i is the terminal valid token and rewards are zero on masked tool-observation tokens. Equation 14 mirrors the implemented order: terminal reward expansion, token-level K1 subtraction, and a reverse discounted cumulative sum. The resulting $A_{i,t}$, rather than a single trajectory-level scalar, weights each optimized token.

The location of the KL estimator matters. Under on-policy sampling, K1 and the nonnegative K3 estimator $\exp(-d^{\text{K1}}) - 1 + d^{\text{K1}}$ estimate the same reverse-KL value, but their optimization gradients are not interchangeable. K1 used as a stop-gradient reward yields the score-function gradient of the KL-regularized policy objective, whereas directly differentiating K3 as an auxiliary loss gives a biased gradient for that objective [Liu et al., 2025, Shah et al., 2025]. Controlled LLM-RL experiments also report stronger and more stable optimization with K1-in-reward than with K3-in-loss [Shah et al., 2025]. We compute K1 from π_{θ^-} and π_{ref} without gradient and optimize it through the reward path, while retaining K3 only as a diagnostic. This reference regularizer is separate from the category-expert distillation term in Section 4.5.

MiniRL token update: behavior correction with a proximal safeguard. The third challenge above arises because the rollout engine records behavior probabilities under μ_b , whereas the trainer optimizes π_{θ} after recomputing the pre-update policy π_{θ^-} . Cross-engine numerical differences and asynchronous policy lag can therefore make these probabilities different. MiniRL corrects this mismatch with the bounded importance weight $w_{i,t}$. In addition, the proximal ratio $\rho_{i,t}^{\text{prox}}$ defines an asymmetric safeguard on the direction of the policy update. For a valid assistant token,

$$\begin{aligned} \rho_{i,t}^{\text{prox}}(\theta) &= \frac{\pi_{\theta}(a_{i,t} \mid h_{i,t})}{\pi_{\theta^-}(a_{i,t} \mid h_{i,t})}, \\ w_{i,t}(\theta) &= \text{clip}\left(\frac{\pi_{\theta}(a_{i,t} \mid h_{i,t})}{\mu_b(a_{i,t} \mid h_{i,t})}, c_{\min}, c_{\max}\right), \\ M_{i,t}(\theta) &= 1 - \mathbf{1}\left[(A_{i,t} > 0 \wedge \rho_{i,t}^{\text{prox}} > 1 + \epsilon_+) \right. \\ &\quad \left. \vee (A_{i,t} < 0 \wedge \rho_{i,t}^{\text{prox}} < 1 - \epsilon_-)\right]. \end{aligned} \quad (15)$$

The sign-aware mask suppresses a token update only after the policy has moved beyond the corresponding boundary in the direction encouraged by its advantage. Truncated importance sampling provides the primary correction for the rollout-trainer probability mismatch, while the mask acts as a proximal safeguard. Section 5 reports the rollout-group size, KL coefficient, and clipping bounds.

Turn-aware reduction and the final objective. The fourth challenge arises from the alternating structure of assistant action turns and tool observations in each SWE trajectory; action turns vary sharply in both count and length. Let $\mathcal{S}_i$ be the set of contiguous valid response segments in trajectory i , each corresponding to one assistant turn. The complete Agentic-miniRL loss directly combines the mask, behavior correction, token advantage, and turn-aware reduction:

$$\mathcal{L}_{\text{Agentic-MiniRL}}(\theta) = - \frac{1}{B_{\text{valid}}} \sum_i \frac{1}{|\mathcal{S}_i|} \sum_{s \in \mathcal{S}_i} \sum_{t \in s} M_{i,t}(\theta) \text{sg}(w_{i,t}(\theta)) \cdot A_{i,t} \log \pi_{\theta}(a_{i,t} \mid h_{i,t}). \quad (16)$$

This is the minimized loss implemented by the trainer. Rewards, advantages, the Boolean mask, and the clipped importance weight are treated as non-differentiated quantities. Its outer reduction is seq-mean-segment-sum: it sums token losses inside each assistant turn, averages turns within a trajectory, and then averages the B_{valid} valid trajectories. Tool-observation tokens are excluded by the response mask. Consequently, neither raw trajectory length nor the number of tool interactions alone determines a trajectory’s outer weight, while a long action retains its token-summed learning signal. We use this recipe for Pooled RL, Balanced RL, and category-expert RL; the implementation and training settings are specified in Section 5.

4.4 Refresh–Repair–Expand training of category experts

We train one expert for each derived category $c \in \{A, B, C\}$. Category separation defines each expert’s scope, but does not by itself ensure strong teachers for integration. The initial experts improve average training-instance success rates, yet these gains coexist with instance-level regressions (Section 6.2). This motivates combining further RL with explicit consolidation of successful trajectories and a refreshed assessment of instance mastery.

A common approach to long-horizon SWE post-training is to synthesize trajectories with a larger teacher, distill them by SFT, and only then apply RL [Song et al., 2026]. Instead, all three experts start from the same strong base policy π_0 and acquire their category behavior directly through executable Agentic RL. This removes an external trajectory teacher from the expert-training loop. Hereafter, *teacher* refers only to the resulting expert used by MOPD in Section 4.5; no external model supplies its solution trajectories or action targets. Teacher-free SWE-agent training is feasible at long horizon [Golubev et al., 2025]. To support expert development under changing instance mastery, we develop **Refresh–Repair–Expand (RRE)**, a policy-adaptive loop that co-evolves each expert and its training frontier. The loop couples exploration of category tasks with reuse of the expert’s existing successful behavior: the updated policy determines which instances receive repair supervision and which tasks enter the next RL phase. Algorithm 1 specifies this process. Each round consists of an RL phase and a Repair SFT phase, with a mastery refresh after each; before another RL round, the repaired policy probes an expanded pool formed by randomly adding previously omitted base-zero and base-one instances.

Signal-aware initial exploration. Let $\mathcal{D}_c$ be the candidate pool for category c . We estimate the mastery of instance x under a policy π with M independent executable rollouts,

$$\hat{p}(x; \pi) = \frac{1}{M} \sum_{j=1}^M R(\tau_j), \quad \tau_j \sim \pi(\cdot \mid x), \quad (17)$$

where $R(\tau) \in \{0, 1\}$ is the verifier outcome. Initial expert training reuses the base-model mastery estimates computed during corpus construction: the category training sets $\mathcal{S}_c^0$ inherit their pass rates from the shared annotated corpus, with no additional initial refresh. The RL selector prioritizes non-extreme pass rates, for which repeated rollouts expose both successes and failures, and admits a small set of zero-pass-rate instances only when the model has produced a verified success for that instance elsewhere. Saturated instances and zero-pass-rate instances without any success evidence

Algorithm 1 Refresh–Repair–Expand training of category experts

Require: Shared base π_0 ; broader category pools $\{\mathcal{D}_c\}$; verifier V ; rounds R

Require: Initial training sets $\{\mathcal{S}_c^0\}$; stored base mastery $\{p_c^0\}$ on the candidate pools

Ensure: Category experts $\{T_A, T_B, T_C\}$

Select(Π, c) = $\arg \max_{\pi \in \Pi} S_c(\pi)$ selects within one training phase.

```
1: for each category  $c \in \{A, B, C\}$  do
2:    $\pi \leftarrow \pi_0; \mathcal{S} \leftarrow \mathcal{S}_c^0; p \leftarrow p_c^0$  ▷ Reuse stored base pass rates
3:    $H \leftarrow \{x : p_c^0(x) > 0\}$ 
4:   for  $r = 1, \dots, R$  do
5:      $p_{\text{pre}} \leftarrow p|_{\mathcal{S}}$ 
6:     Explore and refresh on the current RL training set
7:      $(\Pi_{\text{RL}}, \mathcal{B}^+) \leftarrow \text{AGENTICRL}(\pi, \mathcal{S}, V)$ 
8:      $\pi_{\text{RL}} \leftarrow \text{Select}(\Pi_{\text{RL}}, c)$ 
9:      $p_{\text{RL}} \leftarrow \text{REFRESH}(\pi_{\text{RL}}, \mathcal{S}, V)$ 
10:     $H \leftarrow H \cup \{x : \mathcal{B}^+(x) \neq \emptyset \vee p_{\text{RL}}(x) > 0\}$ 
11:    Repair with successes from this RL phase, then refresh again
12:     $q(x) \leftarrow g_{\text{rep}}(p_{\text{RL}}(x))$  for  $x$  with  $\mathcal{B}^+(x) \neq \emptyset$ 
13:     $\mathcal{D}_{\text{rep}} \leftarrow \text{SAMPLEREPLAY}(\mathcal{B}^+, q)$ 
14:     $\Pi_{\text{SFT}} \leftarrow \text{REPAIRSFT}(\pi_{\text{RL}}, \mathcal{D}_{\text{rep}})$ 
15:     $\pi \leftarrow \text{Select}(\Pi_{\text{SFT}}, c)$ 
16:     $p_{\text{SFT}} \leftarrow \text{REFRESH}(\pi, \mathcal{S}, V)$ 
17:     $H \leftarrow H \cup \{x : p_{\text{SFT}}(x) > 0\}$ 
18:    Record mastery changes  $p_{\text{RL}} - p_{\text{pre}}$  and  $p_{\text{SFT}} - p_{\text{RL}}$ 
19:    if  $r < R$  then
20:      Expand beyond the preceding RL training set
21:       $\mathcal{E} \leftarrow \{x \in \mathcal{D}_c \setminus \mathcal{S} : p_c^0(x) \in \{0, 1\}\}$ 
22:       $\mathcal{U} \leftarrow \mathcal{S} \cup \text{RANDOMSAMPLE}(\mathcal{E})$ 
23:       $p \leftarrow \text{REFRESH}(\pi, \mathcal{U}, V)$ 
24:       $H \leftarrow H \cup \{x : p(x) > 0\}$ 
25:       $\mathcal{S} \leftarrow \text{FRONTIERSELECT}(\mathcal{U}, p, H)$ 
26:    end if
27:  end for
28:   $T_c \leftarrow \pi$  ▷ Best target-category model in the final training phase
29: end for
30: return  $\{T_A, T_B, T_C\}$ 
```

consume little or no exploration budget. This follows the reward-signal motivation behind dynamic sampling [Yu et al., 2025], but tracks per-instance mastery across training phases rather than only filtering all-correct or all-incorrect groups inside an RL update. The selected set $\mathcal{S}_{c, \text{RL}}^{(r)}$ is optimized with the Agentic RL objective in Section 4.3; exact pass-rate strata and budgets are specified in Section 5.

Coupling policy progress with data refresh and replay. The initial category pool is selected using executable pass-rate estimates from the base policy. These estimates characterize a *policy–instance pair*; they are not immutable properties of the instance. Once RL changes the policy, the pass-rate landscape drifts and the original selection gradually ceases to describe where useful training signal lies. Selected instances can become saturated and provide little rollout contrast, base-zero instances can become learnable while remaining outside the fixed pool, and previously solved instances can regress even as aggregate performance improves. These possibilities motivate revisiting both the instance selection and the successful trajectories accumulated during RL.

Online resampling updates which tasks are explored next. RRE couples that choice with a separate supervised replay phase that directly trains on successful behavior from the expert’s earlier rollouts. We therefore pause after each RL phase to reevaluate the selected expert and allocate historical successes to low-mastery instances. We refresh again after Repair SFT because supervised updates also change instance mastery. Reassessing the preceding RL set tracks these changes, while probing the broader candidate pool before the next RL phase finds training signal outside the original selection. These phase boundaries make the updated policy the basis for both repair supervision and the next exploration distribution.

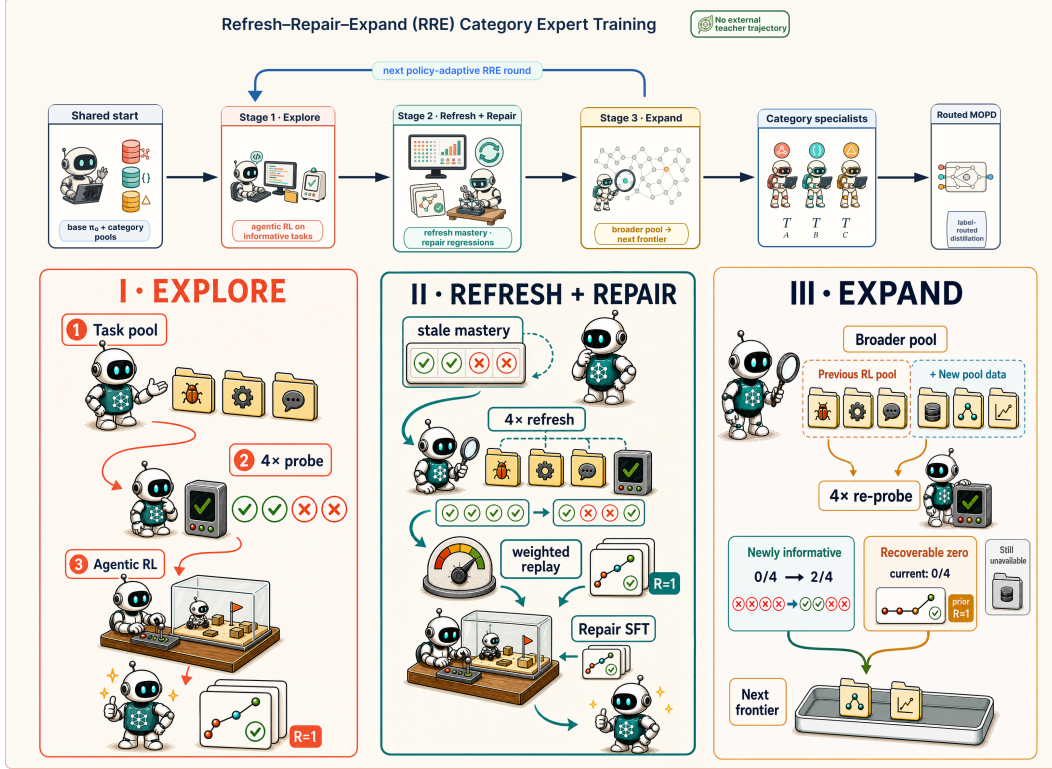


Figure 4: **Refresh-Repair-Expand training of category experts.** The same loop is instantiated independently for categories A, B, and C from a common strong base. Initial selection reuses the base-model probes from corpus construction. Agentic RL explores on an informative category frontier and records the expert’s own verifier-approved trajectories. Mastery is refreshed on the preceding RL training set after both RL and Repair SFT. The post-RL estimates guide successful-trajectory replay; the post-SFT estimates track the repaired policy’s mastery. Before the next RL phase, the selected SFT policy probes an expanded pool that augments the preceding RL set with randomly sampled base-zero and base-one tasks. Solid navy arrows track stage and policy progression, green arrows reuse self-generated verifier-approved trajectories, and dashed control arrows indicate mastery-weighted replay. The blue return path closes the next RRE round. No external trajectory teacher is used.

Refresh: synchronize the mastery map with the policy. Within each training phase, we select the model with the highest target-category resolution S_c among that phase’s evaluated models, using the corresponding Pro subset as specified in Section 5. Denote the selected post-RL and post-SFT policies in round r by $\pi_{c,RL}^{(r)}$ and $\pi_{c,SFT}^{(r)}$, respectively. After RL, we reevaluate $S_{c,RL}^{(r)}$ under $\pi_{c,RL}^{(r)}$ to obtain $\hat{p}_{c,RL}^{(r)}(x)$; let $\hat{p}_{c,pre}^{(r)}(x)$ be the corresponding estimate under the policy entering that RL phase. The paired change

$$\Delta_c^{(r)}(x) = \hat{p}_{c,RL}^{(r)}(x) - \hat{p}_{c,pre}^{(r)}(x) \quad (18)$$

measures changes in estimated instance mastery: aggregate improvement does not imply that every previously solved instance remains solved. The new $\hat{p}_{c,RL}^{(r)}$ also replaces the pre-RL mastery map. The paired estimates locate observed regressions, while current mastery guides replay allocation and future RL data selection.

Repair: consolidate self-generated successes. During the preceding RL phase, the expert has already accumulated a trajectory buffer from its own on-policy rollouts,

$$\mathcal{B}_c^{(r)}(x) = \{\tau : \tau \text{ was generated for } x \text{ in category-}c \text{ RL phase } r, R(\tau) = 1\}. \quad (19)$$

We replay only these verifier-approved trajectories. An instance receives a quota $q_c^{(r)}(x) = g_{rep}(\hat{p}_{c,RL}^{(r)}(x))$, where g_{rep} is non-increasing: low-mastery instances receive more distinct successful

trajectories, whereas currently mastered instances receive fewer. If fewer successful trajectories exist than the quota, they are cycled only to meet the fixed repair budget. The preceding delta is used to diagnose which low-mastery cases are regressions; the actual replay quota is determined by current mastery. A zero-success post-RL probe does not imply an empty historical buffer; instances without any verified successful trajectory in that buffer receive no repair examples. Starting from $\pi_{c,\text{RL}}^{(r)}$, Repair SFT minimizes

$$\mathcal{L}_{c,\text{rep}}^{(r)}(\pi) = - \sum_x \sum_{\tau \in \text{Sample}(B_c^{(r)}(x), q_c^{(r)}(x))} \log \pi(\tau \mid x). \quad (20)$$

Here $\log \pi(\tau \mid x)$ abbreviates $\sum_{t \in \mathcal{I}_{\text{assistant}}(\tau)} \log \pi(y_t \mid h_t)$, where $\mathcal{I}_{\text{assistant}}(\tau)$ indexes only assistant-generated tokens; prompt and tool-observation tokens are excluded from the loss. The Sample operator returns a multiset, so repeated trajectories from cyclic replay contribute with their sampling multiplicities. The highest-scoring model on category c within this SFT phase becomes $\pi_{c,\text{SFT}}^{(r)}$. We then refresh mastery on the same RL training set to obtain $\hat{p}_{c,\text{SFT}}^{(r)}$ and track its change from $\hat{p}_{c,\text{RL}}^{(r)}$. This refresh is performed even after the final repair phase. We call this step *Repair SFT*: it is a reward-filtered form of trajectory self-distillation, not a second source of expert demonstrations. It shares the teacher-free principle of recent self-distillation methods [Hübotter et al., 2026], while using complete, executable SWE trajectories verified by the environment rather than feedback-conditioned token distributions.

Expand: reopen the current learning frontier. Before the next RL phase, we augment $\mathcal{S}_{c,\text{RL}}^{(r)}$ with a random sample of additional instances from $\mathcal{D}_c$ whose base-model pass rates were zero or one. Denote the resulting refresh pool by $\mathcal{U}_c^{(r+1)}$. Sampling these additional instances keeps reevaluation within a bounded budget while revisiting tasks omitted by the initial curriculum: previously unsolved tasks may now yield successes, and previously saturated tasks may no longer be fully mastered. We evaluate $\pi_{c,\text{SFT}}^{(r)}$ on $\mathcal{U}_c^{(r+1)}$ to obtain the next pre-RL mastery map $\hat{p}_{c,\text{pre}}^{(r+1)}$; it complements the post-SFT refresh on the preceding training set. Infrastructure-failed or incomplete refreshes are rerun before an instance can enter the next training set. Let $H_c^{(r)}(x) = 1$ indicate that x has at least one verified success in the base evaluation, an earlier expert evaluation, or the expert’s accumulated RL history. RRE constructs the next exploration set from

$$\mathcal{F}_c^{(r+1)} = \{x \in \mathcal{U}_c^{(r+1)} : 0 < \hat{p}_{c,\text{pre}}^{(r+1)}(x) < 1\}, \quad (21)$$

$$\mathcal{Z}_c^{(r+1)} = \{x \in \mathcal{U}_c^{(r+1)} : \hat{p}_{c,\text{pre}}^{(r+1)}(x) = 0 \wedge H_c^{(r)}(x) = 1\}, \quad (22)$$

$$\mathcal{S}_{c,\text{RL}}^{(r+1)} = \mathcal{F}_c^{(r+1)} \cup \mathcal{Z}_c^{(r+1)}. \quad (23)$$

The first set is the current informative frontier; the second retains currently unsolved instances with evidence of historical success. Pure hard-zero and currently saturated instances are excluded from the main RL budget. Agentic RL then resumes from $\pi_{c,\text{SFT}}^{(r)}$, closing the refresh–repair–expand cycle shown in Figure 4. After the final round, $T_c = \pi_{c,\text{SFT}}^{(R)}$: the expert is the highest-scoring target-category model within the final training phase.

RRE is related to contemporary self-evolution frameworks that alternate experience internalization and policy optimization. In particular, SPEE extracts transferable textual experience from successful and failed reasoning trajectories, distills that experience, and then applies GRPO [Ren et al., 2026]. RRE advances this paradigm to long-horizon agentic SWE and changes the object that evolves: instead of maintaining an abstract global experience pool, it maintains a policy-adaptive, executable task frontier. The refresh is grounded in repeated environment verification, the repair phase reuses the same expert’s successful action trajectories, and expansion is conditioned on both current mastery and historical success. Repeating this loop independently yields T_A , T_B , and T_C , which share an origin and agent interface and are optimized on different category pools. Their stage-wise performance and subsequent integration are evaluated in Section 6.

4.5 Label-routed multi-teacher on-policy distillation

Separate experts provide category-conditioned policies, while deployment calls for a single model. We use the same category map to assign each training task to its corresponding expert during MOPD.

The student is initialized from the original base policy π_0 and trained on a category-balanced mixture of the three initial-RL task pools, formed by upsampling the smaller pools to match the largest. The student π_θ generates the complete trajectory, so teachers are queried only on prefixes the student actually visits, following the on-policy distillation setting of MOPD [Ma et al., 2026]. For each category expert T_k , $k \in \{1, \dots, K\}$, let $m_k(x) \in \{0, 1\}$ be the task-level route and let q_t be the behavior log probability under the rollout policy. On sampled tokens we define two log ratios: the student–teacher gap and the teacher–reference gap,

$$d_{t,k} = q_t - \log \pi_{T_k}(y_t | h_t), \quad e_{t,k} = \log \pi_{T_k}(y_t | h_t) - \log \pi_{\text{ref}}(y_t | h_t). \quad (24)$$

The routed multi-teacher advantage used in our runs includes a ReLU-gated extrapolation term:

$$A_t^{\text{MOPD}} = \sum_{k=1}^K m_k(x) [-d_{t,k} + (\lambda_k - 1) \max(e_{t,k}, 0)]. \quad (25)$$

With $\lambda_k = 1$, the second term vanishes and the advantage reduces exactly to the MOPD imitation term; with $\lambda_k > 1$, the second term extrapolates beyond imitation on tokens assigned higher probability by the teacher than by the reference. MOPD runs in pure distillation mode: each task is rolled out once, the environment return does not enter the student loss, the group baseline and K1 reference penalty are disabled, π_{ref} appears only as the extrapolation anchor, and Equation 25 is the entire advantage signal. Response masks intersect teacher-route masks. The A/B/C mapping is mutually exclusive, so each routed task receives the signal of its corresponding expert; an unrouted sample receives zero advantage. The per-teacher imitation coefficient is one in our configuration.

The student objective. The complete MOPD loss reuses the turn-aware reduction of Section 4.3, with the routed advantage entering as a per-token weight:

$$\mathcal{L}_{\text{MOPD}}(\theta) = -\frac{1}{B_{\text{valid}}} \sum_i \frac{1}{|\mathcal{S}_i|} \sum_{s \in \mathcal{S}_i} \sum_{t \in s} \tilde{A}_t^{\text{MOPD}} \log \pi_\theta(y_t | h_t). \quad (26)$$

Here $\tilde{A}_t^{\text{MOPD}} = \text{clip}(A_t^{\text{MOPD}}, -c_a, c_a)$ is the bounded routed advantage, with finite clip bound $c_a = 5$. Teacher log-probabilities, the behavior log-probability q_t , and the advantage itself are treated as non-differentiated quantities, and only the student log-likelihood is differentiated. The gradient raises the student’s probability of a sampled token when its routed advantage is positive: the imitation component favors tokens the routed teacher assigns more probability than the rollout policy did, and the gated extrapolation component adds a nonnegative advantage contribution on tokens favored by the teacher over the reference. The update is a vanilla policy gradient: unlike Agentic-miniRL, it applies no proximal mask or importance-ratio correction, retaining only the finite advantage clip so that no single token’s student–teacher log-ratio can dominate the token-summed turn reduction. Token losses are summed within each assistant turn, and these turn-level sums are averaged within each trajectory by the outer $1/|\mathcal{S}_i|$ factor.

Reward extrapolation and the reference policy. Standard on-policy distillation pulls the student toward the teacher. We augment this imitation signal with reward extrapolation (ExOPD) [Yang et al., 2026]: the $(\lambda_k - 1) \max(e_{t,k}, 0)$ term in Equation 25 allows the student update to include a reference-anchored extrapolation direction rather than imitation alone. We adopt this formulation from prior work on cross-domain expert integration and apply it to intra-domain, label-routed experts.

The coefficient λ_k is set per teacher. The audited configuration uses $\lambda_k = 1.25$. For π_{ref} , we use the student’s initial policy as the reference; in our same-origin setting the student is itself initialized from that base, so the extrapolation direction remains tied to the policy from which category specialization began. This choice costs one additional frozen reference forward. The integration hyperparameters are reported in Section 5.6.

Rectifying the extrapolation direction. The teacher–reference gap $e_{t,k}$ takes both signs across sampled tokens: positive where the routed teacher assigns more probability than π_{ref} , negative when it assigns less. Without gating, the extrapolation term thus mixes positive and negative contributions, and in our runs these components are comparable in magnitude, so the net extrapolation magnitude remains well below that of the imitation term. We therefore rectify the extrapolation direction with a ReLU gate that retains the positive teacher–reference log-probability gaps,

$$\tilde{e}_{t,k} = \max(e_{t,k}, 0), \quad (27)$$

as already included in Equation 25, without changing the routing, the imitation term, or the aggregation. The gate zeroes only the extrapolation contribution where the routed teacher assigns no higher probability than the reference; the imitation signal remains unchanged. With $\lambda_k \geq 1$, the retained extrapolation contribution is nonnegative on every token and the term acquires a systematic direction instead of acting as a signed residual that cancels across tokens. The ReLU-gated extrapolation is enabled in the audited configuration.

Same-origin teachers and monitoring. The teachers are same-origin category experts. Shared tokenization, prompt format, base checkpoint, and agent interface reduce irrelevant teacher–student mismatch across experts trained on different category pools. Section 6.3 reports training rollout scores and student–teacher KL during integration. Extrapolation raises the number of frozen teacher-side forwards from K to $K+1$.

5 Experimental setup

5.1 Research questions

Our central evaluation asks whether category-aware expert training and integration produce a stronger single SWE policy. Three research questions connect Balanced RL and expert development to this final comparison. **RQ1** examines whether Balanced RL alleviates the uneven category gains observed under Pooled RL, alongside overall resolution. **RQ2** examines the initial category experts, the instance-level learning patterns that motivate further development, and how target-category performance evolves through the RL and repair stages of RRE. **RQ3** evaluates whether label-routed MOPD can consolidate these experts into one policy while retaining their category-specific gains and outperforming Pooled RL and Balanced RL in aggregate resolution and every evaluation category. We use *joint RL* as the umbrella term for training one shared policy across categories. The two Agentic-miniRL baselines are named *Pooled RL* and *Balanced RL*; figures may abbreviate them as *Pooled* and *Balanced*.

5.2 Base model and executable agent

All principal policies start from the same Qwen3.6-27B base model [Qwen Team, 2026]. Training and Pro-618 evaluation use the same R2E-Gym agent scaffold [Jain et al., 2025]: the model receives the issue description and accumulated interaction history, invokes structured repository and shell tools, and submits a patch that is scored by executable fail-to-pass and pass-to-pass tests. A training rollout uses a 131,072-token context, at most 150 agent actions, and at most 12,288 newly generated tokens per action; Agentic RL samples $G=8$ trajectories per task.

5.3 Benchmarks and evaluation configurations

Pro-618 is our primary benchmark for final performance comparison and category-wise analysis. Its evaluation scores also inform checkpoint selection during training. SWE-bench Multilingual provides a complementary evaluation of the fixed final models on an additional benchmark. Table 3 summarizes their task populations and agent frameworks. These evaluation settings are distinct from the training rollout settings above.

Pro-618. Our primary benchmark is an audited 618-instance subset of SWE-bench Pro [Deng et al., 2025], evaluated with the R2E-Gym agent scaffold. The common category mapping yields 221, 201, and 196 instances in Pro-A, Pro-B, and Pro-C. These scores support checkpoint selection, longitudinal category analysis, stage-wise expert evaluation, and the final policy comparison. Base, Pooled RL/Balanced RL, final MOPD, and expert endpoint results report means and population standard deviations over three evaluation rounds; checkpoint sweeps use the separately documented round selections. The selection rules are specified below, and Section 5.4 explains the audited subset construction.

SWE-bench Multilingual. We evaluate the final MOPD policy, the base model, and Pooled RL and Balanced RL on all 300 instances using SWE-agent [Yang et al., 2024]. The category mapping assigns 72, 25, and 176 instances to A, B, and C; the remaining 27 are reported as unassigned and

Table 3: Benchmarks and configured inference budgets for policy evaluation. U denotes instances outside the A/B/C mapping; all instances contribute to Full resolution.

Setting	Pro-618	SWE-bench Multilingual
Instances	618	300
A/B/C/U	221/201/196/0	72/25/176/27
Agent framework	R2E-Gym	SWE-agent
Context limit (tokens)	200,000	200,000
Output limit per call (tokens)	25,600	12,280
Maximum actions/iterations	150	150
Temperature	1.0	1.0
Top- p / top- k	0.95 / 20	0.95 / 20
Trajectories per task per round	1	1
Environment/task timeout (s)	10,000	12,000
Inference precision	BF16	BF16

retained in Full. The available results comprise three evaluation rounds for each reported model; we report mean resolution and population standard deviation across rounds. Pooled RL and Balanced RL use the same Pro-618-selected policies as in the main comparison.

Execution and scoring. Each rollout produces one candidate patch, scored by the benchmark’s executable verifier. Resolution uses the full task denominator for the corresponding benchmark; unsuccessful or missing evaluations receive zero and are not dropped. Comparisons are made within each benchmark and agent framework, rather than between absolute scores from the two benchmarks. Controlled comparisons use matching system prompts, tool schemas, environment images, decoding settings, action and token budgets, timeout policies, and verifiers within each benchmark.

Both evaluation configurations use BF16 vLLM inference with a 200,000-token context limit, temperature 1.0, top- p 0.95, and top- k 20, with one trajectory per task per round. Pro-618 permits up to 25,600 output tokens per call and 150 agent actions, with a 10,000-second environment timeout. Its verifier applies the model patch in a fresh sandbox. Multilingual permits 12,280 output tokens per call and 150 iterations, with a configured 12,000-second task timeout. It evaluates all 300 instances using SWE-agent.

5.4 Pro-618 construction and audit

Our primary category-aware evaluation uses SWE-bench Pro [Deng et al., 2025]. Compared with the bug-fix-dominated SWE-bench Verified set [Jimenez et al., 2024, OpenAI, 2024], Pro spans more varied repository contexts, maintenance intents, and modification scales. The cross-benchmark Labeler profiles in Table 2 quantify this difference: 87.0% of Verified instances are bug fixes and 86.2% are single-file changes, whereas 47.3% of Pro instances are bug fixes and 59.2% require cross-module changes. This breadth makes Pro better suited to measuring category-level learning dynamics within repository-level SWE. Verified is used here to characterize the benchmark’s task profile; the policy-evaluation benchmarks and agent configurations are specified in Section 5.3.

We evaluate on 618 of the 731 SWE-bench Pro instances because defective task specifications or graders confound agent capability with benchmark validity. Recent audits show that specification gaps, incorrect ground truths, and weak executable graders can materially distort measured agent performance [Kim, 2026, OpenAI, 2026]. An underspecified issue can admit several reasonable implementations while the grader accepts only one; a defective grader can reject even the reference patch; and a prompt–test mismatch can score behavior different from what the task requests. Removing these cases makes the longitudinal category curves reflect policy changes rather than known benchmark defects.

We apply the instance-level evidence classes released with the public determinacy audit [Kim, 2026]. Its reproducible receipts identify 83 mechanically established specification/determinacy defects, 26 tasks for which adversarial judgments by two model families identify an underspecified grading choice, and three tasks whose reference patches fail isolated grader replay. We also exclude one prompt–test mismatch documented by the official OpenAI audit [OpenAI, 2026]. The remaining 618 instances form *Pro-618*. Applying the common routing rule yields Pro-A, Pro-B, and Pro-C

with 221, 201, and 196 instances, respectively. Appendix A provides the exclusion accounting and audit-manifest fields.

5.5 Training data and category-conditioned curricula

The executable candidate pool described in Section 4.1 contains approximately 32K instances. Before training-corpus selection, SWE LABELER uses Qwen3.7-max to annotate this entire synthetic pool. These labels are retained for both initial category assignment and subsequent RRE pool expansion. To construct the training corpus, we evaluate the base model on these candidates and select instances with estimated pass rates between 0 and 0.5, focusing training on tasks that the initial policy has not yet reliably mastered. This selection yields 3,048 synthetic instances. We combine them with 3,675 tasks from a four-language slice of SWE-rebench V2 [Badertdinov et al., 2026], forming the 6,723-task corpus used by the Pooled RL and Balanced RL comparisons. Every task has a containerized repository, an issue-style specification, and executable fail-to-pass and pass-to-pass tests.

The SWE-rebench V2 instances are also annotated by SWE LABELER using Qwen3.7-max. The same Domain L1 rule in Section 4.2 maps eligible instances into A (service/data/security), B (user-facing applications), or C (systems/tooling/runtimes). Of the 6,723 instances, 2,769 map to these three categories: 601 to A, 516 to B, and 1,652 to C. The remaining instances fall outside the A/B/C mapping. The Pooled RL see-saw experiment trains one policy on all 6,723 tasks. The stratified Balanced RL run samples 516 tasks from each category, for 1,548 tasks in total. Within this initial 6,723-task corpus, category B contains 516 instances and therefore determines the common per-category budget.

Table 4: **Training data used by the controlled experiments.** A, B, and C are shown in separate columns. Candidate and repairable rows report the support available to each selection step; training and sampled rows report the materialized data. Repair trajectories are sampled only from verifier-approved rollouts generated by the same expert in the immediately preceding RL phase. MOPD repeats A/B task records to match C’s size; source-record counts and upsampled record counts are shown separately.

Phase	Quantity	Category			Total
		A	B	C	
Pooled RL	training instances	—	—	—	6,723
Balanced RL	training instances	516	516	516	1,548
Initial expert RL	training records	601	516	1,652	2,769
First Repair SFT	repairable instances	571	501	1,498	2,570
	sampled preceding-RL trajectories	1,623	1,481	4,384	7,488
Expanded expert RL	candidate instances	1,000	1,000	2,282	4,282
	training instances	566	502	1,435	2,503
Second Repair SFT	repairable rollout keys	549	487	1,371	2,407
	sampled preceding-RL trajectories	1,392	1,245	3,840	6,477
MOPD integration	training records from initial RL pools	601	516	1,652	2,769
	category-balanced training records	1,652	1,652	1,652	4,956

Initial expert training uses these category-specific instances and reuses their stored base-model pass rates without an additional initial mastery refresh. After each RL phase, we reevaluate the selected expert on that phase’s training instances using four fresh executable rollouts per instance. These updated pass rates guide Repair SFT sampling from verifier-approved successful trajectories generated by the same expert during that RL phase. Table 5 allocates more successful trajectories to instances whose current mastery is lower; a trajectory is reused only when an instance has fewer distinct successful rollouts than its target. After Repair SFT, we refresh mastery on the same RL training instances with four fresh rollouts per instance. Before the next RL phase, we additionally evaluate an expanded candidate pool with the selected post-SFT expert to identify newly informative tasks. The next RL set combines the current frontier with historically supported recoverable-zero instances, following Section 4.4.

After the first repair, we augment each category’s initial RL training set with randomly sampled instances from the already labeled 32K synthetic pool that had base-model pass rates of zero or one and were omitted from initial RL. The additions are assigned using their existing labels and

the same Domain L1 rule; no new labeling is required. The resulting refresh pools contain 1,000, 1,000, and 2,282 instances for A/B/C, respectively. The selected post-SFT experts reevaluate these pools to identify changes in mastery and new training signal among previously excluded tasks. Thus the broader refresh uses sampled additions rather than rescoring the full 32K synthetic candidate pool. From these 4,282 candidates, the expanded-stage selection materializes 566/502/1,435 training instances, as reported in Table 4. After this expanded RL phase, a second mastery refresh materializes 1,392/1,245/3,840 verifier-approved repair trajectories for A/B/C.

Table 5: **Mastery-adaptive sampling for Repair SFT.** Mastery is measured with four fresh executable rollouts. Each selected trajectory must be a verifier-approved success previously generated by the same expert.

Current successes	Target trajectories	Sampling emphasis
0 or 1 of 4	4	strongest repair pressure
2 of 4	3	unstable frontier
3 of 4	2	consolidation
4 of 4	1	retention

For MOPD, we balance and combine the three experts’ Stage 1 RL task pools, retaining their A/B/C labels for teacher routing. The source pools contain 601/516/1,652 records. We upsample A and B by repeating task records to match C’s 1,652 records, yielding 4,956 training records with equal category counts. This changes sampling frequency without adding new tasks; the source pools contain 2,769 training records before upsampling. The student starts from the original Qwen3.6-27B base rather than a post-RRE expert. Consequently, the stored pre-Stage 1 base-model pass rates also characterize the student’s initial mastery, providing the same basis for reusing this selected task pool without an additional initial refresh. Training uses fresh student-generated trajectories on these tasks, not the experts’ stored Repair SFT trajectories.

5.6 Training infrastructure and optimization

Agentic RL. We implement all RL phases in ROLL, which separates distributed policy optimization, model inference, environment interaction, and reward computation [Wang et al., 2025b]. We use the agent framework from R2E-Gym [Jain et al., 2025], with ROLL executing each trajectory in a ROCK sandbox provisioned from the task’s repository image; the sandbox isolates file-system changes and process failures while exposing the shell, repository tools, and executable verifier to the agent [Wang et al., 2025c]. Each sandbox receives 4 CPU cores and 16 GiB of memory, with a 9,600-second trajectory limit.

The initial and expanded RL phases use one common optimization configuration for A, B, and C (Table 6). ROLL allocates 32 devices to policy optimization and 32 to vLLM rollout generation. Generation and optimization may overlap with an asynchrony bound of one policy update. Within an RL phase, the task manifest is fixed: dynamic resampling is disabled and tasks are traversed without replacement within each epoch. The mastery refreshes and curriculum changes described in Section 4.4 therefore occur between phases.

Repair SFT. After each RL phase, each selected expert is fully fine-tuned on its own verifier-approved repair trajectories. The loss is applied to assistant response tokens while user, system, and tool-observation context remains conditioning input. We use the same 131,072-token limit as RL, Adam with a cosine schedule, and bf16 training. A/B/C use effective batches of 64/64/128, and each repair phase runs for approximately five corpus epochs. All other SFT settings are shared across categories and repair phases.

MOPD integration. The student is trained by pure label-routed on-policy distillation with the three final RRE experts frozen. Each rollout batch contains 128 tasks with one trajectory per task, followed by one policy epoch, using learner micro-batch size 1 and gradient accumulation of 64. We use a learning rate of 3×10^{-6} , weight decay 0.01, a cosine schedule with no warmup, bf16 precision, and gradient-norm clipping at 20. All three teachers use imitation coefficient 1 and extrapolation coefficient $\lambda_k = 1.25$ with the ReLU gate enabled; the token-level routed advantage is clipped to $[-5, 5]$. The student uses the turn-aware vanilla policy-gradient objective in Equation 26, without

Table 6: **Core training configurations.** RL settings are shared by the initial and expanded phases and by all three category experts. Category-valued SFT entries are ordered A/B/C.

Setting	Agentic RL	Repair SFT
Objective	AGENTIC-MINIRL with group-relative advantages	full-parameter, response-only cross-entropy
Context	131,072 tokens; 150 actions; 12,288 generated tokens/action	131,072 tokens
Batch	16 tasks/update, $G=8$; 128 trajectories	micro-batch 1, accumulation 16; effective batch 64/64/128
Optimization	$\text{lr } 3 \times 10^{-7}$; weight decay .01; cosine; 20 warmup steps	Adam, $\text{lr } 5 \times 10^{-6}$; weight decay .01; cosine; 5% warmup
Stabilization	$(\epsilon_{\text{low}}, \epsilon_{\text{high}}) = (.20, .27)$; importance ratio $[0, 5]$; KL coefficient .001; grad norm 20	bf16; grad norm 1
Generation	temperature 1.0; top- p .95; top- k 20	—
Parallelism	32 train + 32 inference devices; train TP8/CP2; vLLM TP4	TP1/CP8/PP4; world size 128/128/256
Updates	one policy epoch per rollout batch	approximately five corpus epochs per repair phase

an environment-reward term, additional KL loss, or entropy bonus. Context, action, and per-action token limits match expert RL, as do the decoding temperature, top- p , and top- k . The upsampled task records are shuffled and traversed without replacement; repeated copies of a task remain separate sampling entries. The training seed is 42. We allocate 32 devices each to student optimization and rollout generation, 16 to each teacher, and 16 to the frozen reference, with an asynchrony bound of one policy update. The integration sandbox timeout is 7,200 seconds.

5.7 Reward integrity and anti-hacking protocol

An executable success is meaningful only when the submitted source change, rather than an unintended side effect, causes the required tests to pass. We distinguish two failure modes that are often conflated. *Verifier exploitation* changes or bypasses the tests, harness, or execution state so that an incorrect solution receives reward. *Solution leakage* recovers the upstream fix from future Git history or an external repository. The latter may produce a functionally correct patch, but it does not measure the agent’s ability to solve the supplied task. Both channels are realistic in long-horizon coding RL [Baker et al., 2025, Zhao et al., 2026, Rajan, 2026]. Public SWE tasks also exhibit memorization-sensitive performance, making it important to distinguish genuine repository reasoning from recovered task artifacts [Liang et al., 2025a, Badertdinov et al., 2025].

An audit of unrestricted pilot trajectories exposed the characteristic behaviors behind these risks: agents queried `git log -all`, read future objects with `git show`, added or fetched upstream remotes, and searched public hosting services for the corresponding fix. We consequently apply the defense-in-depth protocol in Table 7. The controls operate at different boundaries: task construction removes textual answer leakage; repository sanitization restricts the rollout state; deferred tests and fresh-sandbox replay protect the verifier; and trajectory auditing catches external answer retrieval that a repository-local control cannot see.

For ordinary training instances, repository sanitization resets the worktree to the declared base revision, removes upstream refs and non-current branches, deletes tags not ancestral to that revision, expires reflogs, and prunes unreachable objects. Retaining ancestral tags avoids breaking projects whose build systems derive version metadata from Git. The stricter Pro-618 policy removes all remotes, non-current branches, tags, reflogs, and unreachable objects in the agent sandbox, while preserving image-built dependencies and runtime files. A failed sanitization invalidates the episode.

The evaluator never scores the mutable rollout workspace. It extracts the agent’s source patch, removes unsupported binary hunks and known image-runtime artifacts, starts a fresh sandbox from the same instance image, resets it to the base revision, applies only the sanitized patch, and then injects the held-out test patch and executes the verifier. Thus edits to visible tests, shell state, installed packages, or long-lived processes cannot carry into the scoring environment. Complete trajectories are additionally scanned for target-repository search, clone/fetch, raw patch download, and exact fix-SHA access. Confirmed external answer retrieval is reported as an integrity failure, assigned a score of zero, and retained in the evaluation denominator.

Table 7: Defense-in-depth protocol for preserving reward integrity. Each control closes a different path from an executable pass to a misleading training or evaluation signal.

Risk surface	Control	Integrity condition
Problem-statement leakage	Separate fix and test patches; reject generated statements that reveal source/test paths or patch-added code.	The agent receives the issue-style specification and base repository, not the reference implementation or held-out test patch.
Future repository state	Reset to the base revision; remove remotes, non-current branches, post-base tags, reflogs, and unreachable objects. Pro-618 removes all tags in the rollout sandbox.	Future fixes cannot be recovered through local <code>git log</code> , <code>git show</code> , tags, or dangling objects.
Verifier and environment tampering	Defer held-out test injection; export only a sanitized source patch and replay it in a fresh instance sandbox.	Reward depends on the replayed patch under evaluator-controlled tests, not on rollout-side files, processes, packages, or test edits.
External answer retrieval	Inspect the full action trajectory for target-repository search, clone/fetch, patch/raw downloads, and exact fix-SHA access.	Confirmed retrieval is an integrity failure, receives a score of zero, and remains in the fixed evaluation denominator.

5.8 Baselines and comparison settings

Pooled RL and the category see-saw. To answer RQ1, we evaluate the common base and every scheduled checkpoint of the Pooled RL 6,723-task run on Pro-618. We report the overall curve together with Pro-A/B/C, rather than selecting only the best aggregate checkpoint. The 1,548-task stratified Balanced RL run repeats the same analysis with 516 training instances per category.

For the Pooled RL endpoint in Table 12, we select the checkpoint with the highest mean Pro-618 Full resolution among checkpoints with three valid evaluation rounds. We report Full and Pro-A/B/C means and standard deviations from those same three rounds of that checkpoint. For Balanced RL, we select one of the checkpoints tied for the highest Full resolution in the single-round sweep, fix that checkpoint, and report its mean and standard deviation across three evaluation rounds, including the initial screening round.

Expert formation and RRE. To answer RQ2, A, B, and C experts share the same initialization and training algorithm but receive their route-specific curricula. We evaluate each candidate checkpoint once on the corresponding Pro-A, Pro-B, or Pro-C subset within each RL or Repair SFT phase, and select the checkpoint with the highest target-category resolution to enter the next phase. We then fix the selected checkpoint and conduct repeated evaluations; the stage table reports the mean and standard deviation over three evaluation rounds. The final expert is selected within the last training phase by the same rule. These category scores guide model selection and performance reporting; training-instance pass rates are estimated separately to guide repair-trajectory sampling and the next RL curriculum. Table 11 reports each expert’s target-category resolution after initial RL, the first Repair SFT, expanded RL, and the second Repair SFT.

Training-instance diagnostics. We align base-model, post-RL, and post-Repair SFT success-rate estimates on matched records from the initial experts’ training sets: 601 for A, 516 for B, and 1,652 for C. The post-RL checkpoints are the selected initial experts. The post-SFT estimates come from the mastery refresh before expanded RL; newly probed tasks are excluded from this matched-instance comparison. Actual repair-training membership identifies which previously regressed instances received SFT examples. Each instance’s rate uses its actual number of sampled attempts, typically four, and category means give each training record equal weight. Repeated records share their instance-level success-rate estimate. These training-set estimates diagnose observed changes in instance mastery, separately from Pro-618 checkpoint selection and performance reporting. Appendix D details duplicate handling and the paired-snapshot protocol.

Expert integration. To answer RQ3, the MOPD student is compared with the common base, Pooled RL and Balanced RL, and category-routed experts. The routed expert system selects the corresponding expert for each category. The MOPD comparison therefore tests how much of the expert improvement is recovered in one policy. The student is initialized from the common base, the same checkpoint from which the category experts began.

5.9 Metrics, uncertainty, and failure accounting

Final policy comparisons report overall and category-wise resolution. *Full* denotes the overall benchmark score in tables and figures; *Full gain* and *overall gain* denote the same score difference. On Pro-618, Full gain, minimum category gain G_{sim} , and SSG summarize improvement over the common base, following Equations 5 and 6. Table 8 averages these metrics over the 50 displayed checkpoints per method and reports final Full gain separately. The process metrics are computed from per-checkpoint two-round mean scores in Figure 5. Its compressed evaluation-index axis preserves checkpoint order within each run, rather than aligning elapsed training steps across runs. Table 13 compares the final single policies using their three-round mean scores. Category-wise trajectories show the opposing movements that characterize the temporal see-saw. Endpoint integration is also evaluated by the aggregate and category lifts defined below.

For a candidate policy π , we measure its category lift over the Pooled RL policy π_{pooled} as

$$d_c(\pi) = S_c(\pi) - S_c(\pi_{\text{pooled}}), \quad d_{\min}(\pi) = \min_{c \in \{A, B, C\}} d_c(\pi). \quad (28)$$

We call $d_{\min}$ the *minimum category lift over Pooled RL*. Positive $d_c(\pi_{\text{MOPD}})$ for all three categories constitutes a category-wise Pareto improvement over Pooled RL; $d_{\min}$ reports the weakest of these three improvements as a single, directly interpretable number. We report the corresponding overall lift separately using the official Pro-618 aggregate.

We also compute the three category lifts and their minimum relative to Balanced RL by replacing the comparator in Equation 28.

Expert formation is summarized by stage-wise target-category scores, training rollout curves, and matched instance-level mastery changes. Integration is summarized by category-expert gain recovery. For target expert T_c ,

$$\text{Recovery}_c = \frac{S_c(\pi_{\text{MOPD}}) - S_c(\pi_{\text{base}})}{S_c(\pi_{T_c}) - S_c(\pi_{\text{base}})}. \quad (29)$$

When T_c does not improve over the base, Recovery is undefined rather than using a non-positive denominator.

Checkpoint sweeps are longitudinal diagnostics. Final endpoint comparisons use repeated executable rollout rounds, retain paired task identities, and report mean resolution and population standard deviation across three rounds, with the task denominators stated. For the Full-score differences on SWE-bench Multilingual, we additionally report paired instance-bootstrap 95% intervals. Each model’s three scores for an instance are averaged first; the 300 paired instances are then resampled with replacement 10,000 times, and the 2.5th and 97.5th percentiles of the mean differences give the intervals. Resolution uses a fixed task denominator. Unsuccessful or missing evaluations and integrity violations receive a score of zero and remain in the denominator.

6 Results and analysis

The main outcome is the performance of the integrated single policy relative to Pooled RL and Balanced RL. To interpret that comparison, we first examine the effect of balanced sampling on category-level learning. We next trace the evolution of the three RRE experts and evaluate how much of their category-specific improvement is retained by a single MOPD policy.

6.1 Category dynamics under Pooled RL and Balanced RL

One way to use task categories is to balance the data composition of a shared training stream. To answer RQ1, we compare Pooled RL with training on a category-balanced subset of 1,548 instances (516 per category), stratified by base-model pass rate. The comparison extends the observations in

Section 3 by examining whether a more balanced training mixture alleviates the observed category see-saw. Guided by Table 1, we examine whether Balanced RL yields higher Full gain, higher minimum category gain G_{sim} , and lower SSG than Pooled RL.

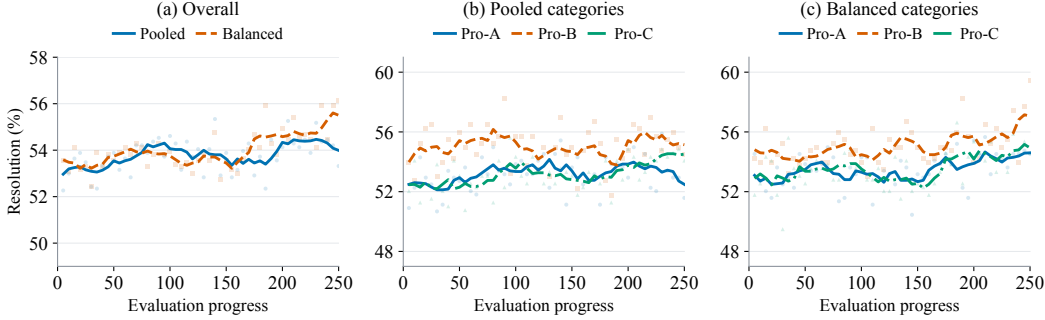


Figure 5: **Category dynamics under Pooled RL and Balanced RL.** Panel (a) compares Pro-618 Full resolution; panels (b) and (c) show the corresponding Pro-A/B/C trajectories on identical category-score axes. Faint points show two-round mean evaluation scores at each checkpoint. Lines show five-point centered moving averages. Category scores continue to exhibit opposing movements under Balanced RL.

Figure 5(a) shows similar overall performance across the two training runs, despite Balanced RL using only 1,548 distinct training instances compared with 6,723 for Pooled RL. The endpoint results in Table 12 likewise show close Full resolution (55.34% versus 55.50%). Balanced RL also shows a local advantage on Pro-B during parts of training, and its final model has a slightly higher three-round mean Pro-B resolution (57.05% versus 56.72%). Thus, in this setting, a smaller training set focused on the target categories can retain competitive performance.

Table 8 distinguishes final-model performance from training-process summaries. Relative to the same base-model Full score of 52.64%, the final Pooled RL model gains 2.86 percentage points and the final Balanced RL model gains 2.70 points, using their three-round evaluation means. Both training recipes therefore produce substantial overall improvement. The lower training-process Full gains, 1.13 and 1.37 points respectively, average across the 50 displayed checkpoints per method rather than measuring final-model improvement.

The process summaries assess whether gains are sustained across categories. Balanced RL has higher average Full gain (1.37 versus 1.13 points) and minimum category gain G_{sim} (0.19 versus 0.08 points) than Pooled RL. Its average SSG is also higher (1.18 versus 1.05 points). The minimum-gain averages refer to the weakest category gain at each evaluated checkpoint, not to the category scores of the final models. Thus, balancing improves both average overall and minimum category gains, but does not reduce the aggregate-to-minimum gap; it does not jointly improve all three quantities sought in Table 1. Thus, the process-level limitation concerns the consistency of category gains, not an absence of overall learning in the final models.

Table 8: **Category-gain summaries under Pooled RL and Balanced RL.** Training-process columns average per-checkpoint metrics over the 50 displayed checkpoints per method, using two-round mean scores as in Figure 5. The last column reports the final models’ three-round mean Full gain, using the same models as Table 12. All gains use the same three-round base-model reference; values are in percentage points.

Training mixture	Training-process mean			Final Full gain ↑
	Full gain ↑	G_{sim} ↑	SSG ↓	
Pooled RL	1.13	0.08	1.05	+2.86
Balanced RL	1.37	0.19	1.18	+2.70

The trajectories reveal a related temporal pattern. Figure 5(c) still contains opposing category movements: an improvement in one category can coincide with a regression in another. Balanced RL can improve all three categories at individual points, but their gains are not consistently sustained

together over training. Taken together, these results answer RQ1: Balanced RL maintains close overall performance with fewer distinct training instances and raises average minimum category gain, but does not reduce the average gap or eliminate opposing category movements. This motivates examining a second way to use the category structure: developing experts in separate training streams before integrating them into one policy.

6.2 Category-expert evolution through RRE

To answer RQ2, we first assess the initial category experts and diagnose their instance-level learning. We then examine how RRE develops these experts before their subsequent integration by MOPD.

Does initial category-specific RL suffice? The initial experts improve their target-category Pro-618 scores over the base model, reaching 53.54%, 55.06%, and 52.55% on A, B, and C, respectively (Table 11). These means remain below the Pooled RL endpoint’s 54.75%, 56.72%, and 55.10%. To investigate why initial category-specific RL yields only modest target-category gains, we examine how success rates change across individual training instances.

Average progress coexists with instance-level regressions. Table 9 follows the same matched training records from the base model through initial RL and Repair SFT. During initial RL, mean success rates increase by 7.26, 6.76, and 4.60 points for A, B, and C. Yet across 2,769 training records, observed rates increase on 1,108, decrease on 839 (30.3%), and remain unchanged on 822 (Table 10, Total rows). Thus, average progress within a category does not imply uniform improvement across its training instances. These are changes between two sampled estimates, not continuous training trajectories or per-instance tests of forgetting.

Table 9: Training-record success rates through initial RL and first Repair SFT. Scores are equal-weight means over the same matched training records (%). The final two columns report SFT gains in percentage points.

Expert	N	Base	Initial RL	Repair SFT	$\Delta_{\text{SFT-RL}}$	$\Delta_{\text{SFT-Base}}$
A	601	38.79	46.05	56.28	+10.23	+17.49
B	516	38.78	45.54	59.59	+14.05	+20.81
C	1,652	38.68	43.28	52.75	+9.47	+14.07

This uneven learning motivates the consolidation and refresh steps of RRE. Repair SFT reuses verifier-approved successes accumulated during the preceding RL phase, allocating more replay to currently low-mastery instances with available trajectories. A low post-RL estimate can therefore coexist with historical successes available for replay.

Recovery and gain retention after repair. The post-SFT refresh shows higher mean success rates in all three training categories: 56.28%, 59.59%, and 52.75% for A, B, and C, respectively. These exceed the initial-RL rates by 10.23, 14.05, and 9.47 points and the base rates by 17.49, 20.81, and 14.07 points (Table 9). Of the 839 records whose observed rates decreased during initial RL, 751 were included in repair training: 179 for A, 141 for B, and 431 for C. Of these, 125 (69.8%), 103 (73.0%), and 264 (61.3%) recover to or above their respective base rates; another 7, 5, and 30 partially recover (Table 20). Thus, the aggregate improvement is accompanied by recovery on many of the previously regressed instances actually included in SFT.

Relative to the same base reference, the number of higher-scoring records increases from 259 to 336 for A, 216 to 314 for B, and 633 to 877 for C; the corresponding lower-scoring counts decrease from 191 to 104, 146 to 81, and 502 to 377 (Table 10). Thus, all three categories show both broader gains and fewer observed losses after repair. The magnitude decomposition in Appendix D likewise shows larger positive contributions and smaller negative contributions relative to Base.

Recovery coexists with further changes in instance mastery. Across all 2,769 training records, 1,211 rates increase after SFT, 978 remain unchanged, and 580 decrease. Of the 1,108 records that initially improved over Base, 404 lose some of their post-RL gains; 135 still exceed Base and 183 return to the base rate, leaving 86 below it. Hence, 78.7% of this 404-record cohort remain at or above Base despite their decline from the RL endpoint. By category, these proportions are 81.6% (84/103) for A,

Table 10: Training-record gains and losses relative to the base model. Both stages use the same matched training records within each category. Higher/equal/lower count observed pass rates relative to Base, not changes from the immediately preceding stage. Parentheses in the Repair SFT rows show the change in each count relative to Initial RL.

Expert	Stage	N	Higher	Equal	Lower
A	Initial RL	601	259	151	191
A	Repair SFT	601	336 (+77)	161 (+10)	104 (−87)
B	Initial RL	516	216	154	146
B	Repair SFT	516	314 (+98)	121 (−33)	81 (−65)
C	Initial RL	1,652	633	517	502
C	Repair SFT	1,652	877 (+244)	398 (−119)	377 (−125)
Total	Initial RL	2,769	1,108	822	839
Total	Repair SFT	2,769	1,527 (+419)	680 (−142)	562 (−277)

80.0% (60/75) for B, and 77.0% (174/226) for C. These observations show substantial recovery and positive net gains without uniform improvement on every instance. They also motivate refreshing mastery after SFT: the updated estimates, together with probes of the expanded candidate pool, guide the next RL curriculum. To assess whether the benefits extend beyond the training instances, we relate these recovery results to the corresponding target-category Pro-618 evaluations below.

Learning within the RL stages. Figure 6 shows the training process in the initial and expanded RL stages. Comparing mean raw rollout scores in the early and late portions of the displayed trajectories, initial RL improves A/B/C by 7.76/12.74/6.58 percentage points; expanded RL improves them by 9.53/3.65/2.15 points. All three experts therefore make progress within each stage over the shared window, although the trajectories fluctuate and include local regressions. These within-stage changes are measured on each expert’s own training stream; the two stages use different task pools.

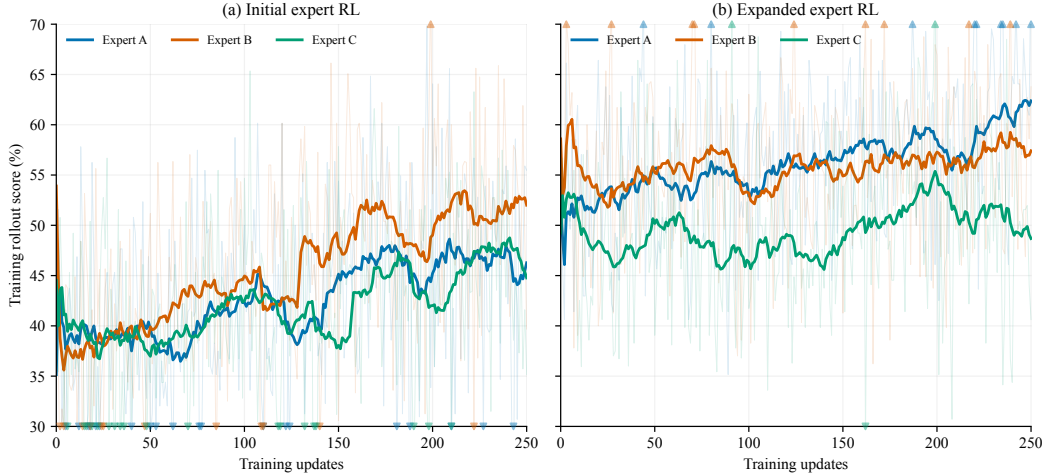


Figure 6: **Category-expert training dynamics in the two RL stages.** Panels (a) and (b) show training rollout scores for initial RL (Stage 1) and expanded RL (Stage 3), respectively. Faint traces show raw scores; bold traces show 20-update trailing means, with shorter windows at the start. Both panels use the same 30–70% vertical range; boundary triangles mark raw scores outside this range. Each expert is trained on its own category pool, which is refreshed between stages.

Stage-wise target-category gains. Table 11 follows each expert through the four stages. Each column evaluates an expert on its corresponding category, using the stage-selected checkpoints described in Section 5. The base-model evaluation provides the reference row.

Table 11: Stage-wise RRE expert results. Entries report target-category resolution (%) as mean $\pm$ population standard deviation across three evaluation rounds. Parentheses below each entry give the mean gain over the base model in percentage points.

Training stage	Expert A / Pro-A	Expert B / Pro-B	Expert C / Pro-C
Base	51.73 $\pm$ 0.43	54.39 $\pm$ 1.17	51.87 $\pm$ 1.27
Initial RL	53.54 $\pm$ 0.77 (+1.81)	55.06 $\pm$ 0.62 (+0.66)	52.55 $\pm$ 1.10 (+0.68)
First Repair SFT	57.32 $\pm$ 0.43 (+5.58)	56.55 $\pm$ 0.62 (+2.16)	55.44 $\pm$ 0.96 (+3.57)
Expanded RL	58.67 $\pm$ 1.19 (+6.94)	58.37 $\pm$ 2.00 (+3.98)	56.97 $\pm$ 0.96 (+5.10)
Second Repair SFT	59.58 $\pm$ 1.40 (+7.84)	58.87 $\pm$ 1.31 (+4.48)	57.48 $\pm$ 0.64 (+5.61)

The recovery observed on training instances is accompanied by improved performance on the corresponding Pro-618 categories. Following the first repair phase, mean target-category resolution increases over initial RL by 3.77, 1.50, and 2.89 percentage points for A, B, and C, respectively. Together with the higher training success rates and fewer below-base instances in Tables 9 and 10, these results support the role of repair in expert development: recovery on training tasks is accompanied by category-level gains beyond those tasks.

The selected checkpoints show increasing mean target-category resolution across the four stages for all three experts. Expanded RL yields further gains, followed by smaller mean improvements after the second repair. By the end of the chain, the experts exceed the base model on their corresponding categories by 7.84, 4.48, and 5.61 points. The gains also extend beyond the first RRE round: compared with the first repair endpoint, the second repair endpoint improves mean resolution by 2.26, 2.32, and 2.04 points for A, B, and C, respectively. The first repair is followed by the largest adjacent-stage gain, with further improvements observed after expanded RL and the second repair. After expanded RL, all three target-category means exceed the Pooled RL endpoint; the final experts’ margins over that reference are 4.83, 2.16, and 2.38 points on A, B, and C, respectively. The advantage thus emerges through expert development rather than from the initial category RL phase alone.

Together, the instance-level diagnostics, within-stage learning curves, and stage-wise Pro-618 results answer RQ2. Initial category-specific RL leaves uneven progress across training instances; repair recovers many observed losses while raising mean target-category resolution, and refreshed curricula and expanded RL continue expert development. The next section asks whether these expert gains can be retained in one integrated policy, rather than only in a category-routed expert system.

6.3 Integrating expert gains into a single policy

To answer RQ3, the final comparison asks whether the improvements developed by separate experts can be retained in a single deployed model.

Learning during multi-teacher integration. Figure 7 tracks the student’s training rollout score and student–teacher KL during MOPD. Comparing the mean raw values in the early and late portions of training, rollout score increases from 45.51% to 56.91%, a gain of 11.41 percentage points, while mean KL decreases by 43.0%. The declining KL indicates closer alignment with the routed teachers on student-generated training contexts, accompanied by stronger task performance on the category-balanced training stream. These concurrent trends connect teacher alignment to the student’s improving training performance; we next evaluate the integrated policy on the benchmarks.

Overall and category-wise benchmark performance. Table 12 compares MOPD with the base model, Pooled RL and Balanced RL, and the category-routed experts. **Category-routed experts** denotes a three-model reference that uses Expert A on Pro-A, Expert B on Pro-B, and Expert C on Pro-C. This row reports the experts before single-model integration, whereas MOPD reports the integrated single policy.

The final MOPD policy achieves 58.04% mean Full resolution across three evaluation rounds, improving over the base model by 5.39 percentage points. Its Pro-A/B/C scores are 58.07%, 59.37%,

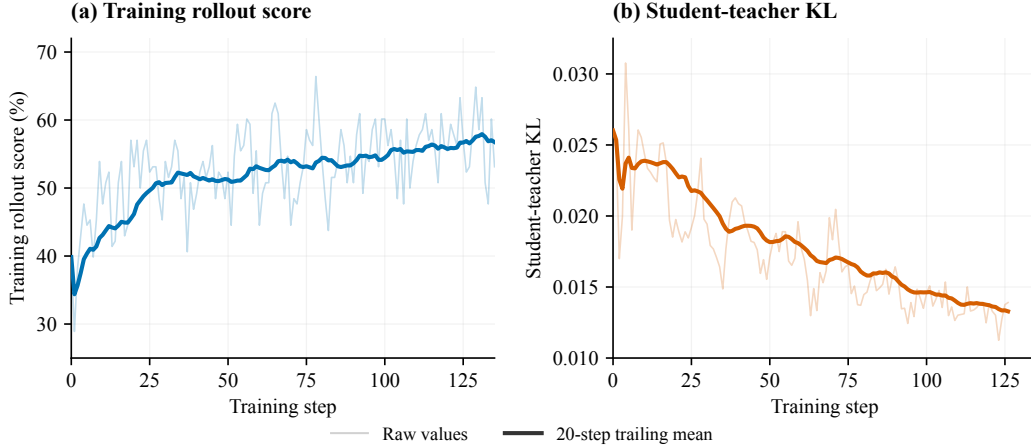


Figure 7: **Learning during multi-teacher on-policy distillation.** Panel (a) shows the student’s training rollout score; panel (b) shows student–teacher KL. Faint traces show raw values and bold traces show 20-step trailing means, with shorter windows at the start. Teacher alignment improves alongside the student’s training task performance.

Table 12: Overall and category-wise resolution on Pro-618 (%). Base, Pooled RL/Balanced RL, and MOPD scores, and category-wise entries of the routed expert reference, report mean $\pm$ population standard deviation across three evaluation rounds. Parentheses below each non-base score give the change from the corresponding base-model mean in percentage points, computed before rounding. The category-routed experts report the corresponding expert’s result in each category before single-model integration.

Policy	Full $\uparrow$	Pro-A $\uparrow$	Pro-B $\uparrow$	Pro-C $\uparrow$
Base model	52.64 ± 0.28	51.73 ± 0.43	54.39 ± 1.17	51.87 ± 1.27
Pooled RL	55.50 ± 0.46 (+2.86)	54.75 ± 1.33 (+3.02)	56.72 ± 1.07 (+2.32)	55.10 ± 1.44 (+3.23)
Balanced RL	55.34 ± 0.92 (+2.70)	54.45 ± 0.85 (+2.71)	57.05 ± 0.94 (+2.65)	54.59 ± 1.25 (+2.72)
Category-routed experts (3 models)	—	59.58 ± 1.40 (+7.84)	58.87 ± 1.31 (+4.48)	57.48 ± 0.64 (+5.61)
MOPD	58.04 ± 0.20 (+5.39)	58.07 ± 1.13 (+6.33)	59.37 ± 1.02 (+4.98)	56.63 ± 1.10 (+4.76)

and 56.63%, corresponding to gains of 6.33, 4.98, and 4.76 points over Base. Compared with Pooled RL, MOPD improves Full resolution by 2.54 points and A/B/C by 3.32, 2.65, and 1.53 points. Compared with Balanced RL, the corresponding improvements are 2.70 points overall and 3.62, 2.32, and 2.04 points by category. Thus, the higher aggregate score is accompanied by higher mean resolution in every category, with minimum category lifts of 1.53 and 2.04 points over Pooled RL and Balanced RL, respectively (Equation 28).

Which expert gains survive integration? Figure 8 compares the final models’ category gains relative to the same base policy. MOPD retains 80.8%, 111.1%, and 84.8% of the corresponding expert gains on A, B, and C, respectively, using Equation 29. Recovery above 100% on B means that the student exceeds its corresponding expert’s mean score, by 0.50 points. On A and C, the student remains 1.51 and 0.85 points below the corresponding experts, while still outperforming both joint-RL baselines. Integration therefore preserves enough of the developed expert gains to improve all three categories over both joint-RL baselines, with the lowest proportional recovery on A.

Overall gain, minimum category gain, and gain balance. Table 13 compares these three aspects for the final single policies, using the same base reference. Unlike the training-process averages in Table 8, these metrics are computed from each final model’s three-round mean benchmark scores.

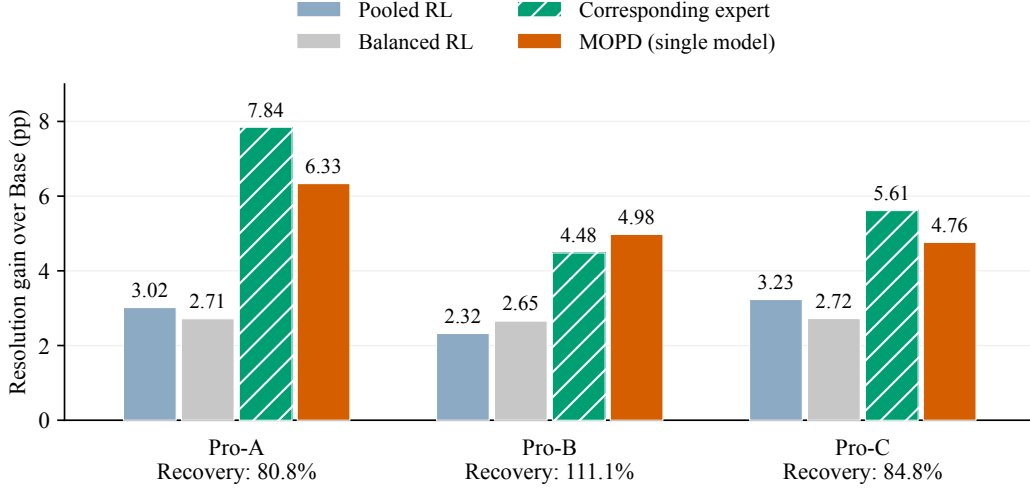


Figure 8: **Category gains before and after single-model integration.** Bars show three-round mean resolution gains over the same base model. Each expert is evaluated on its corresponding category; MOPD uses one integrated model for all categories. Recovery below each group is the fraction of the corresponding expert’s gain retained by MOPD (Equation 29). Round standard deviations are reported in Table 12.

Table 13: **Final-policy gains on Pro-618.** Metrics are computed from three-round mean scores relative to the same base model, in percentage points. G_{sim} is the minimum A/B/C gain; SSG is Full gain minus G_{sim} . Bold marks the best value among the three single policies for each metric.

Policy	Full gain $\uparrow$	G_{sim} $\uparrow$	SSG $\downarrow$
Pooled RL	2.86	2.32	0.54
Balanced RL	2.70	2.65	0.04
MOPD	5.39	4.76	0.63

MOPD achieves the largest Full gain (5.39 points) and minimum category gain G_{sim} (4.76 points). Relative to Pooled RL and Balanced RL, the minimum category gain increases by 2.44 and 2.11 points, respectively. Together with the positive A/B/C differences in Table 12, this shows that the aggregate improvement extends to every category, including the category with the smallest gain over Base.

Relative to Pooled RL, MOPD nearly doubles both the overall and minimum category gains, while SSG increases only slightly, from 0.54 to 0.63 percentage points. Compared with Balanced RL, MOPD achieves higher overall and minimum category gains, although the gains are less uniform (SSG: 0.63 versus 0.04 points). Under the interpretation in Table 1, this is shared improvement with a wider gap, rather than aggregate improvement accompanied by a stagnant or declining minimum gain. The A/B/C scores in Table 12 further confirm that MOPD improves every category over both baselines. These results distinguish a higher level of improvement across categories from a more uniform distribution of gains.

Additional evaluation on SWE-bench Multilingual. Table 14 evaluates the final MOPD policy with SWE-agent on an additional benchmark. MOPD achieves 59.00% mean resolution, compared with 56.22% for the base model, an increase of 2.78 percentage points. Its three observed round scores are 58.67%, 58.67%, and 59.67%; the base scores are 57.00%, 56.33%, and 55.33%. The category breakdown shows gains of 6.48 points on A and 8.00 points on B, while C has the same mean resolution. The 27 unassigned instances also improve by 6.17 points and remain included in the aggregate score. Thus, the observed improvement over the base model extends beyond Pro-618, with the additional gains concentrated in A, B, and the unassigned group rather than distributed uniformly across categories.

Table 14: Resolution on SWE-bench Multilingual with SWE-agent (%). Full reports mean $\pm$ population standard deviation across three evaluation rounds per reported model; category entries are cross-round means. Each round contains the same 300 instances. A/B/C contain 72/25/176 instances; U denotes the 27 unassigned instances, included in Full.

Policy	Full $\uparrow$	A $\uparrow$	B $\uparrow$	C $\uparrow$	U $\uparrow$
Base model	56.22 $\pm$ 0.68	44.91	56.00	61.74	50.62
Pooled RL	55.56 $\pm$ 0.83	47.69	53.33	58.52	59.26
Balanced RL	57.00 $\pm$ 1.19	48.61	56.00	59.66	62.96
MOPD	59.00 $\pm$ 0.47	51.39	64.00	61.74	56.79

Pooled RL achieves 55.56% mean resolution. MOPD exceeds it by 3.44 percentage points overall and by 3.70, 10.67, and 3.22 points on A, B, and C, respectively, while scoring 2.47 points lower on unassigned instances. Balanced RL achieves 57.00% mean resolution. MOPD exceeds it by 2.00 points overall and by 2.78, 8.00, and 2.08 points on A, B, and C, respectively, while scoring 6.17 points lower on unassigned instances. Thus, MOPD achieves higher mean aggregate resolution and higher mean scores in all three routed categories than both joint-RL baselines on this additional benchmark. These results complement the primary Pro-618 evaluation by showing that the advantage over both joint-RL baselines also appears beyond Pro-618 under a common SWE-agent scaffold. Paired instance-bootstrap 95% intervals for the Full differences are [0.33, 5.22] points versus base, [0.67, 6.22] versus Pooled RL, and $[-0.78, 4.78]$ versus Balanced RL, computed from each model’s mean score across its three selected rounds.

Summary of single-policy integration. Taken together, the results answer RQ3 and connect expert development to the final deployed policy. Initial category-specific RL yields modest target-category gains; RRE develops stronger experts, and MOPD retains much of their improvement in one model. The resulting policy achieves higher mean resolution than both joint-RL baselines in aggregate and in all three categories on both benchmarks. On Pro-618, higher Full and minimum category gains coexist with uneven expert-gain recovery. A retains the smallest fraction of its expert’s gain (80.8%), identifying further A-category gain retention as a concrete opportunity to improve integration. This remaining gap coexists with positive A-category lifts over both joint-RL baselines.

7 Discussion

When should one domain be decomposed? A shared task domain need not imply uniform learning demands. Within repository-level software engineering, tasks differ in their engineering contexts, tool interactions, and verification requirements. Decomposition becomes worth exploring when these differences form interpretable task groups and pooled training exhibits persistent differences in their learning trajectories that aggregate performance obscures. In our experiments, category gains move in opposing directions during parts of training, and balancing the training mixture does not clearly remove this pattern. These observations motivate investigating category-specific learning as an alternative to changing mixture proportions alone. They frame decomposition as a hypothesis to evaluate, rather than an assumed advantage of specialization.

Developing experts beyond category separation. Category separation changes which tasks a policy learns from, but does not by itself make those tasks fully learnable or preserve previously acquired behavior. Our initial-RL diagnostics show that average training improvement can coexist with declining success rates on individual instances. This suggests that expert development must address not only differences across categories, but also changing mastery within each category. RRE responds to this second problem by revisiting the expert’s own successful trajectories and refreshing task selection as the policy changes. The observed training-instance recovery and successive target-category improvements support treating specialization as an iterative development process, rather than a one-time partition of the training pool.

Higher performance does not imply uniformly retained gains. Single-model integration should be assessed both by the gains it delivers over the joint-RL baselines and by how much of each expert’s improvement it retains. These perspectives reveal different aspects of the outcome. MOPD

improves overall resolution and every category over both joint-RL baselines, while achieving a higher minimum category gain. Yet its gains remain uneven, and the fraction of expert improvement retained differs across categories. In particular, C has the smallest gain over Base, whereas A has the lowest expert-gain recovery. Thus, the category with the smallest final improvement is not necessarily the one with the largest proportional integration loss. This distinction helps identify whether further progress should focus on developing stronger experts or transferring their existing capabilities more effectively.

8 Limitations

SWE tasks can span multiple categories, whereas our hard-routing configuration assigns each task to one expert category. This simplifies overlapping task structure for category-conditioned training. Training multiple experts and running teacher forwards adds substantial compute, while the best K may vary with data, base model, and budget. Same-origin teachers may be too similar to add signal or, after heavy specialization, too distant for stable integration. Benchmark pass rates do not fully measure patch quality, and test-driven rewards can invite evaluator hacking or reflect brittle task specifications [Baker et al., 2025, Zhao et al., 2026, Rajan, 2026]. Repository sanitization and fresh-sandbox replay prevent local-history leakage and rollout-state tampering, while trajectory audits screen external retrieval; these controls do not eliminate pretraining memorization or every weak-verifier failure. Any result on Pro-618 cannot be generalized to an unexecuted full Pro set. Finally, the experiments evaluate the complete training framework and its stage-wise outcomes; they do not isolate the contributions of each optimization component, replay choice, or routing granularity. Single-run category trajectories also include evaluation variability, and broader comparisons of sampling and routing strategies remain open.

Societal and release considerations. More capable coding agents can improve developer productivity but can also produce insecure changes, automate vulnerability discovery, or create patches whose tests pass without satisfying maintainability requirements. Release decisions should include provenance, licenses, benchmark contamination checks, unsafe-code handling, and safeguards proportionate to model capability.

9 Conclusion

We develop a category-aware expert-training and policy-integration framework for heterogeneous repository-level SWE tasks. SWE Labeler provides evidence-grounded task categories; Agentic-miniRL and RRE develop same-origin experts through executable-reward learning, mastery refresh, and replay of their own successful trajectories; label-routed MOPD consolidates the experts into one deployed policy. The evaluation connects Pooled RL and Balanced RL, expert development, and single-model integration through overall and per-category resolution. The final policy reaches 58.04% mean resolution on Pro-618 and 59.00% on SWE-bench Multilingual, with higher mean resolution than both Pooled RL and Balanced RL overall and in each of the three task categories on both benchmarks. Expert-gain recovery on Pro-618 remains uneven across categories, identifying room to improve single-model integration. This framework treats task categories as units for organizing post-training, with the practical goal of retaining expert gains in a single SWE agent.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes, 2023. URL <https://arxiv.org/abs/2306.13649>.
- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs, 2024. URL <https://arxiv.org/abs/2402.14740>.
- Huan ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-MOPD: Diagnosing and fixing capability imbalance in multi-teacher on-policy distillation, 2026. URL <https://arxiv.org/abs/2608.19098>.

- Association for Computing Machinery. ACM Computing Classification System, 2012 Version, 2012. URL <https://www.acm.org/publications/class-2012>.
- Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents, 2025. URL <https://arxiv.org/abs/2505.20411>.
- Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, and Alexander Golubev. SWE-rebench V2: Language-agnostic SWE task collection at scale, 2026. URL <https://arxiv.org/abs/2602.23866>.
- Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation, 2025. URL <https://arxiv.org/abs/2503.11926>.
- Ben Bogin, Kejuan Yang, Shashank Gupta, Kyle Richardson, Erin Bransom, Peter Clark, Ashish Sabharwal, and Tushar Khot. SUPER: Evaluating agents on setting up and executing tasks from research repositories. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 12622–12645, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.702. URL <https://aclanthology.org/2024.emnlp-main.702/>.
- Florian Brand and Jean-Stanislas Denain. What skills does SWE-bench Verified evaluate?, 2025. URL <https://epoch.ai/publications/what-skills-does-swe-bench-verified-evaluate>. Epoch AI, published 2025-06-13.
- Conventional Commits Contributors. Conventional commits 1.0.0, n.d. URL <https://www.conventionalcommits.org/en/v1.0.0/>. Accessed 2026-08-04.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks?, 2025. URL <https://arxiv.org/abs/2509.16941>.
- Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Agentic reinforced policy optimization, 2025. URL <https://arxiv.org/abs/2507.19849>.
- Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley Professional, 2 edition, 2018. ISBN 9780134757599. URL <https://www.pearson.com/en-us/subject-catalog/p/refactoring-improving-the-design-of-existing-code/P200000000254/9780134757599>.
- Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Sergey Abramov, Andrei Andriushchenko, Filipp Fisin, Sergei Skvortsov, and Boris Yangel. Training long-context, multi-turn software engineering agents with reinforcement learning, 2025. URL <https://arxiv.org/abs/2508.03501>.
- Paul A. Grassi, James L. Fenton, Elaine M. Newton, Ray A. Perlner, Andrew R. Regenscheid, William E. Burr, Justin P. Richer, Naomi B. Lefkowitz, Jamie M. Danker, Yee-Yin Choong, Kristen K. Greene, and Mary F. Theofanos. Digital identity guidelines: Authentication and lifecycle management. Technical Report NIST SP 800-63B, National Institute of Standards and Technology, 2017. Includes updates through 2020; withdrawn in 2025 and superseded by SP 800-63B-4.
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: On-policy distillation of large language models, 2023. URL <https://arxiv.org/abs/2306.08543>.

- Felipe Hoffa. Making sense of the metadata: Clustering 4,000 Stack Overflow tags with BigQuery k-means. [Stack Overflow Blog](#), 2019.
- Chuxuan Hu, Yuxuan Zhu, Antony Kellermann, Caleb Biddulph, Suppakit Waiwitlikhit, Jason Benn, and Daniel Kang. Breaking barriers: Do reinforcement post-training gains transfer to unseen domains? In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=mVhN0veUd>.
- Jonas Hübötter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement learning via self-distillation, 2026. URL <https://arxiv.org/abs/2601.20802>.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic, 2022. URL <https://arxiv.org/abs/2212.04089>.
- International Organization for Standardization. ISO/IEC/IEEE 14764:2022: Software engineering—software life cycle processes—maintenance, 2022. URL <https://www.iso.org/standard/80710.html>. Third edition.
- International Organization for Standardization. ISO/IEC 25010:2023: Systems and software engineering—systems and software quality requirements and evaluation (SQuaRE)—product quality model, 2023. URL <https://www.iso.org/standard/78176.html>. Second edition.
- International Software Testing Qualifications Board. Certified Tester Foundation Level syllabus, version 4.0, 2023. URL <https://www.istqb.org/certifications/certified-tester-foundation-level-ctfl-v4-0/>.
- Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents, 2025. URL <https://arxiv.org/abs/2504.07164>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Kabir Khandpur, Kilian Lieret, Carlos E. Jimenez, Ofir Press, and John Yang. SWE-bench Multilingual. SWE-bench official benchmark release, 2025. URL <https://www.swebench.com/multilingual.html>. Accessed 2026-09-18.
- June Kim. A determinacy audit of SWE-bench Pro: How much of the public set pins the behavior it grades, June 2026. URL <https://github.com/kimjune01/swebench-pro-audit/tree/354b3c02a2e055fcd332488600d251338c841380>.
- Sunghwan Kim, Junhee Cho, Beong woo Kwak, Taeyoon Kwon, Liang Wang, Nan Yang, Xingxing Zhang, Furu Wei, and Jinyoung Yeo. On training large language models for long-horizon tasks: An empirical study of horizon length. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=PnHfrCMKtp>.
- Patrick Tser Jern Kon, Archana Pradeep, Ang Chen, Alexander P. Ellis, Warren Hunt, Zijian Wang, John Yang, and Samuel Thompson. SWE-Protégé: Learning to selectively collaborate with an expert unlocks small language models as software engineering agents, 2026. URL <https://arxiv.org/abs/2602.22124>.
- Hwiwon Lee, Ziqi Zhang, Hanxiao Lu, and Lingming Zhang. SEC-bench: Automated benchmarking of LLM agents on real-world software security tasks. In *Advances in Neural Information Processing Systems 38*, 2025. doi: 10.52202/085713-3878. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/a9168f1c54e5147027f1e8cf83e1a775-Abstract-Conference.html.
- Margaret Li, Suchin Gururangan, Tim Dettmers, Mike Lewis, Tim Althoff, Noah A. Smith, and Luke Zettlemoyer. Branch-train-merge: Embarrassingly parallel training of expert language models, 2022. URL <https://arxiv.org/abs/2208.03306>.

- Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. The SWE-Bench illusion: When state-of-the-art LLMs remember instead of reason, 2025a. URL <https://arxiv.org/abs/2506.12286>.
- Xize Liang, Lin Yang, Jie Wang, Yiyang Lu, Runyu Wu, Hanzhu Chen, and Jianye Hao. Boosting multi-domain fine-tuning of large language models through evolving interactions between samples. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 37427–37441, 2025b. URL <https://proceedings.mlr.press/v267/liang25q.html>.
- Xize Liang, Lin Yang, Jie Wang, Rui Liu, Yang Lu, Jinliang Zeng, Hanzhu Chen, Dong Li, and Jianye Hao. Boosting multi-domain reasoning of LLMs via curvature-guided policy optimization. In *International Conference on Learning Representations*, pages 148814–148846, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/f0bb0cc7cd2a027cff5f237f28def7eb-Paper-Conference.pdf.
- Kezhao Liu, Jason Klein Liu, Mingtao Chen, and Yiming Liu. Rethinking KL regularization in RLHF: From value estimation to gradient optimization, 2025. URL <https://arxiv.org/abs/2510.01555>.
- Nikolai Ludwig, Wasi Uddin Ahmad, Somshubra Majumdar, and Boris Ginsburg. From SWE-ZERO to SWE-HERO: Execution-free to execution-based fine-tuning for software engineering agents, 2026. URL <https://arxiv.org/abs/2604.01496>.
- Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. MOPD: Multi-teacher on-policy distillation for capability integration in LLM post-training, 2026. URL <https://arxiv.org/abs/2606.30406>.
- Chenlin Ming, Chendi Qu, Qizhi Pei, Zhuoshi Pan, Yu Li, Xiaoming Duan, Lijun Wu, and Conghui He. IDEAL: Data equilibrium adaptation for multi-capability language model alignment. In *International Conference on Learning Representations*, pages 7407–7425, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/0ced4d30a9e2d50312991c6cd131a9f5-Paper-Conference.pdf.
- MITRE Corporation. CWE-682: Incorrect calculation, 2026a. URL <https://cwe.mitre.org/data/definitions/682.html>. Common Weakness Enumeration, version 4.20; accessed 2026-08-04.
- MITRE Corporation. CWE-697: Incorrect comparison, 2026b. URL <https://cwe.mitre.org/data/definitions/697.html>. Common Weakness Enumeration, version 4.20; accessed 2026-08-04.
- OpenAI. Introducing SWE-bench Verified, August 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>.
- OpenAI. Separating signal from noise in coding evaluations, July 2026. URL <https://openai.com/index/separating-signal-from-noise-coding-evaluations/>.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym, 2025. URL <https://arxiv.org/abs/2412.21139>.
- Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27B dense model. Hugging Face model card, April 2026. URL <https://huggingface.co/Qwen/Qwen3.6-27B>.
- Shreshth Rajan. Auditing reward hackability in code RL training environments, 2026. URL <https://arxiv.org/abs/2606.16062>.
- Shijie Ren, Xiting Wang, Meng Li, Yujie Guo, Yunhang Yao, Ziheng Peng, Xunlong Wang, Yuetan Chen, Haoyang Zhou, Yunlong Liang, and Fandong Meng. Self-improving large language models via progressive experience evolution, 2026. URL <https://arxiv.org/abs/2608.02139>.

- Semantic Versioning Contributors. Semantic versioning 2.0.0, n.d. URL <https://semver.org/spec/v2.0.0.html>. Accessed 2026-08-04.
- Vedant Shah, Johan Obando-Ceron, Vineet Jain, Brian Bartoldson, Bhavya Kailkhura, Sarthak Mittal, Glen Berseth, Pablo Samuel Castro, Yoshua Bengio, Esmeralda S. Whitamner, Moksh Jain, Siddarth Venkatraman, and Aaron Courville. A comedy of estimators: On KL regularization in RL training of LLMs, 2025. URL <https://arxiv.org/abs/2512.21852>. Revised August 2026.
- Manish Shetty, Naman Jain, Jinjian Liu, Vijay Kethanaboyina, Koushik Sen, and Ion Stoica. GSO: Challenging software optimization tasks for evaluating SWE-agents. In *Advances in Neural Information Processing Systems* 38, 2025. doi: 10.52202/085713-5054. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/ddd98389004161943e646322a16348c3-Abstract-Datasets_and_Benchmarks_Track.html.
- Huatong Song, Lisheng Huang, Shuang Sun, Jinhao Jiang, Ran Le, Daixuan Cheng, Guoxin Chen, Yiwen Hu, Zongchao Chen, Yiming Jia, Wayne Xin Zhao, Yang Song, Tao Zhang, and Ji-Rong Wen. SWE-Master: Unleashing the potential of software engineering agents via post-training, 2026. URL <https://arxiv.org/abs/2602.03411>.
- Sainbayar Sukhbaatar, Olga Golovneva, Vasu Sharma, Hu Xu, Xi Victoria Lin, Baptiste Rozière, Jacob Kahn, Daniel Li, Wen tau Yih, Jason Weston, and Xian Li. Branch-train-mix: Mixing expert LLMs into a mixture-of-experts LLM, 2024. URL <https://arxiv.org/abs/2403.07816>.
- John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2): 257–285, 1988. doi: 10.1207/s15516709cog1202_4.
- Chaofan Tao, Jierun Chen, Yuxin Jiang, Kaiqi Kou, Shaowei Wang, Ruoyu Wang, Xiaohui Li, Sidi Yang, Yiming Du, Jianbo Dai, Zhiming Mao, Xinyu Wang, Lifeng Shang, and Haoli Bai. SWE-Lego: Pushing the limits of supervised fine-tuning for software issue resolving, 2026. URL <https://arxiv.org/abs/2601.01426>.
- Haoran Wang, Zhenyu Hou, Yao Wei, Jie Tang, and Yuxiao Dong. SWE-Dev: Building software engineering agents with training and inference scaling, 2025a. URL <https://arxiv.org/abs/2506.07636>.
- Junlin Wang, Federico Bianchi, Shang Zhu, Fan Nie, Yongchan Kwon, Bhuwan Dhingra, and James Zou. Automated benchmark auditing for AI agents and large language models, 2026. URL <https://arxiv.org/abs/2605.26079>.
- Ruiyi Wang and Prithviraj Ammanabrolu. A practitioner’s guide to multi-turn agentic reinforcement learning, 2025. URL <https://arxiv.org/abs/2510.01132>.
- Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, Zichen Liu, Haizhou Zhao, Dakai An, Lunxi Cao, Qiyang Cao, Wanxi Deng, Feilei Du, Yiliang Gu, Jiahe Li, Xiang Li, Mingjie Liu, Yijia Luo, Ziheng Liu, Yadao Wang, Pei Wang, Tianyuan Wu, Yanan Wu, Yuheng Zhao, Shuaibing Zhao, Jin Yang, Siran Yang, Yingshui Tan, Huimin Yi, Yuchi Xu, Yujin Yuan, Xingyao Zhang, Lin Qu, Wenbo Su, Wei Wang, Jiamang Wang, and Bo Zheng. Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library, 2025b. URL <https://arxiv.org/abs/2506.06122>.
- Weixun Wang, XiaoXiao Xu, Wanhe An, Fangwen Dai, Wei Gao, Yancheng He, Ju Huang, Qiang Ji, Hanqi Jin, Xiaoyang Li, Yang Li, Zhongwen Li, Shirong Lin, Jiashun Liu, Zenan Liu, Tao Luo, Dilxat Muhtar, Yuanbin Qu, Jiaqiang Shi, Qinghui Sun, Yingshui Tan, Hao Tang, Runze Wang, Yi Wang, Zhaoguo Wang, Yanan Wu, Shaopan Xiong, Binchen Xu, Xander Xu, Yuchi Xu, Qipeng Zhang, Xixia Zhang, Haizhou Zhao, Jie Zhao, Shuaibing Zhao, Baihui Zheng, Jianhui Zheng, Suhang Zheng, Yanni Zhu, Mengze Cai, Kerui Cao, Xitong Chen, Yue Dai, Lifan Du, Tao Feng, Tao He, Jin Hu, Yijie Hu, Ziyu Jiang, Cheng Li, Xiang Li, Jing Liang, Xin Lin, Chonghuan Liu, ZhenDong Liu, Zhiqiang Lv, Haodong Mi, Yanhu Mo, Junjia Ni, Shixin Pei, Jingyu Shen, XiaoShuai Song, Cecilia Wang, Chaofan Wang, Kangyu Wang, Pei Wang, Tao Wang, Wei Wang, Ke Xiao, Mingyu Xu, Tiange Xu, Nan Ya, Siran Yang, Jianan Ye, Yaxing Zang, Duo Zhang, Junbo Zhang, Boren Zheng, Wanxi Deng, Ling Pan, Lin Qu, Wenbo Su, Jiamang Wang, Wei

- Wang, Hu Wei, Minggang Wu, Cheng Yu, Bing Zhao, Zhicheng Zheng, and Bo Zheng. Let it flow: Agentic crafting on rock and roll, building the ROME model within an open agentic learning ecosystem, 2025c. URL <https://arxiv.org/abs/2512.24873>.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents, 2025d. URL <https://arxiv.org/abs/2407.16741>.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution, 2025. URL <https://arxiv.org/abs/2502.18449>.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents, 2024. URL <https://arxiv.org/abs/2407.01489>.
- Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. SWE-Fixer: Training open-source LLMs for effective and efficient GitHub issue resolution. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 1123–1139, 2025. doi: 10.18653/v1/2025.findings-acl.62. URL <https://aclanthology.org/2025.findings-acl.62/>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida Wang, and Ofir Press. SWE-bench Multimodal: Do AI systems generalize to visual software domains? In *The Thirteenth International Conference on Learning Representations*, 2025a. URL <https://openreview.net/forum?id=riTiq3i21b>.
- John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents, 2025b. URL <https://arxiv.org/abs/2504.21798>.
- Wenkai Yang, Weijie Liu, Ruobing Xie, Kai Yang, Saiyong Yang, and Yankai Lin. Learning beyond teacher: Generalized on-policy distillation with reward extrapolation, 2026. URL <https://arxiv.org/abs/2602.12125>.
- Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-policy context distillation for language models, 2026. URL <https://arxiv.org/abs/2602.12275>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Liang Zeng, Yongcong Li, Yuzhen Xiao, Changshi Li, Chris Yuhao Liu, Rui Yan, Tianwen Wei, Jujie He, Xuchen Song, Yang Liu, and Yahui Zhou. Skywork-SWE: Unveiling data scaling laws for software engineering in LLMs, 2025. URL <https://arxiv.org/abs/2506.19290>.
- Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. SWE-bench goes live!, 2025. URL <https://arxiv.org/abs/2505.23419>.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1592–1604, 2024. doi: 10.1145/3650212.3680384. URL <https://doi.org/10.1145/3650212.3680384>.

Bingchen Zhao, Dhruv Srikanth, Yuxiang Wu, and Zhengyao Jiang. SpecBench: Measuring reward hacking in long-horizon coding agents, 2026. URL <https://arxiv.org/abs/2605.21384>.

Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, and Junyang Lin. Stabilizing reinforcement learning with LLMs: Formulation and practices, 2025. URL <https://arxiv.org/abs/2512.01374>.

A Evaluation benchmark construction and audit

Pro-618 is an audit-filtered subset of the 731 SWE-bench Pro instances. Benchmark audits have shown that specification gaps, incorrect reference solutions, and incomplete grading logic can change measured performance and even model rankings [Kim, 2026, OpenAI, 2026]. We therefore remove only cases backed by released instance-level evidence.

Table 15: Construction of the Pro-618 evaluation subset. Counts are mutually exclusive at the final exclusion stage.

Construction stage	Instances
Full SWE-bench Pro	731
Public audit: mechanically evidenced specification/determinacy issue	−83
Public audit: two-model-family underspecification finding	−26
Public audit: reference patch fails isolated grader replay	−3
Official OpenAI audit: prompt–test mismatch	−1
Pro-618	618

The first three exclusion groups follow the pinned public determinacy-audit artifact [Kim, 2026]; the final case follows the official OpenAI audit [OpenAI, 2026]. We report the source audit and excluded-instance count for each exclusion group. The accompanying manifest records task membership and exclusion reasons. Applying the common Domain L1 routing rule after filtering gives 221 Pro-A, 201 Pro-B, and 196 Pro-C instances.

B SWE Labeler and category construction

B.1 Label inventory

This appendix records the final label axes used in our experiments. The experiment artifact exports the exact machine-readable definitions, active prompts, and configuration used to assign data; each dataset manifest additionally records the labeling model and source export.

Task Type L1 labels (26). bug-fix, performance-fix, compatibility-fix, feature, enhancement, backward-compatibility, from_scratch, test-add, test-fix, coverage, debug-support, refactor, style, deprecation, build-fix, ci-fix, deployment, packaging, config, user-docs, design-spec, diagram, code-explanation, security-fix, auth, and privacy. The current definition file contains 119 Task Type L2 labels.

Domain L1 labels (21). web_frontend, mobile_dev, desktop_gui, web_backend, database_storage, data_science, data_engineering, os_system, iot_embedded, devops_infra, media_graphics, game_dev, devtools_test, lang_runtime, pkg_manager_cli, cms_ecommerce, blockchain_web3, security_auth, docs_knowledge, non_coding, and other. The current definition file contains 108 Domain L2 labels.

Every L2 entry records its name, source attribution, source-derived definition, operational definition, observable signals, and boundary notes. Some L1 labels intentionally have no L2 children; the runtime returns `unspecified` instead of inventing one.

B.2 Taxonomy grounding and structural audit

Source selection by role. Table 16 summarizes how external material is used. Normative standards stabilize established software-engineering distinctions; curated catalogs provide named defect or transformation concepts; empirical developer metadata suggests coarse domain neighborhoods; and official framework documentation supplies ecosystem-specific meaning. None of these sources is treated as a ready-made universal taxonomy for SWE-agent training.

Table 16: Source families used to ground the label inventory.

Axis / family	Representative sources	Role in SWE LABELER
Task Type: defect and security	MITRE CWE entries and NIST guidance [MITRE Corporation, 2026a,b, Grassi et al., 2017]	Name recurrent failure mechanisms and separate neighboring mechanisms.
Task Type: maintenance and change	ISO/IEC 25010, ISO/IEC/IEEE 14764, Semantic Versioning, and Conventional Commits [International Organization for Standardization, 2023, 2022, Semantic Versioning Contributors, n.d., Conventional Commits Contributors, n.d.]	Anchor quality, maintenance, compatibility, feature, build, CI, and change-intent families.
Task Type: test and refactor	ISTQB testing terminology and Fowler’s refactoring catalog [International Software Testing Qualifications Board, 2023, Fowler, 2018]	Supply established test activities and behavior-preserving transformation concepts.
Repository Do-main	Stack Overflow tag co-occurrence clusters plus standards and official ecosystem documentation [Hoffa, 2019]	Seed coarse technology neighborhoods, then define repository-purpose and framework-specific boundaries.
Scope, complexity, and time	Repository topology, Epoch AI’s SWE-bench analysis, and cognitive-load theory [Brand and Denain, 2025, Sweller, 1988]	Define a project-specific scope rule and adapt empirical brackets or theoretical inspiration into SWE decision rules.

Representative evidence chains. The following examples instantiate Equation 9. In each case, a source concept is translated into evidence available to the offline annotator and separated from its nearest alternatives.

Logic error. The `bug-fix.logic_error` label uses CWE-682 and CWE-697 to anchor incorrect calculations and comparisons. The SWE rule accepts an incorrect condition, arithmetic rule, comparator, or algorithmic result. Array/loop boundary mistakes instead map to `off_by_one`, while unintended mutation maps to `state_corruption`.

API-signature refactoring. For `refactor.api_signature_refactor`, Fowler’s API, algorithm, and data-structure refactorings provide the source concept [Fowler, 2018]. The operational rule covers a changed signature, parameter object, equivalent algorithm, or representation while preserving behavior. Merely creating a helper is `extract_method`; an identifier-only change is `rename_symbol`.

Scientific and numerical software. The `data_science.scientific_numerical` label uses ACM CCS mathematical software [Association for Computing Machinery, 2012] and official scientific- library documentation anchor numerical, symbolic, array, geometry, and domain-science library internals. Training or serving a learned model uses an ML label; feature preparation on top of these libraries is `data_preprocessing`.

Identity lifecycle. For `security_auth.identity_lifecycle`, NIST digital-identity guidance supplies the authenticator-lifecycle anchor [Grassi et al., 2017]. The project rule covers enrollment, renewal, revocation, session, credential, and device-trust lifecycle work. Token-issuance protocols map to `oauth_oidc`, authorization policy to `rbac_acl`, and primitive implementation to `crypto_impl`.

Cross-module scope. The `scope.cross_module` label is a project-defined repository-topology rule, with benchmark task-size analysis as motivational context [Brand and Denain, 2025]. It requires changes across top-level packages or architectural layers within one repository. Several files in one module remain `multi_file_same_module`; independent repositories are `cross_repo`.

Global reasoning. For `complexity.global_reasoning`, high element interactivity is theoretical inspiration [Sweller, 1988]; the SWE rule requires tracing data flow, call chains, or state across components. Choosing among valid architectures is instead `design_decision`. This is an adaptation, not a claim that cognitive-load theory directly specifies SWE labels. Likewise, the Stack Overflow clustering is an empirical seed for domain organization rather than a gold annotation set.

Final admission and boundary protocol. Every label in the frozen inventory satisfies four requirements: (i) it is not already covered by an existing operational definition; (ii) it represents one coherent concept rather than a narrow implementation variant; (iii) it has an attributable source, observable signals, and an explicit non-overlap boundary; and (iv) sparse or heterogeneous residuals can abstain through unspecified. Consequently, related API transformations share `api_signature_refactor`, and enrollment, renewal, revocation, session, credential, and device-trust operations share `identity_lifecycle`. The inventory does not include broad catch-alls such as a generic business-app backend label or a separate label for every individual refactoring operation.

Schema-completeness audit. In the final inventory, all 239 decision records—119 Task Type L2 labels, 108 Domain L2 labels, and 12 orthogonal levels—contain the six schema fields described around Equation 9. All 26 Task Type L1 families, 21 Domain L1 families, and three orthogonal dimensions record a source framework. This audit verifies the completeness of label definitions and their traceability to source concepts.

Orthogonal axes. Modification `scope` has four levels: `single_file`, `multi_file_same_module`, `cross_module`, and `cross_repo`. Cognitive complexity has four levels: `mechanical`, `local_reasoning`, `global_reasoning`, and `design_decision`. Estimated time has four levels: `trivial_15min`, `small_1h`, `medium_4h`, and `large_4h_plus`.

Auxiliary Phase-1 metadata. The implementation records three fields outside the semantic label vector. `code_language` chooses one of 21 normalized values (including `other` and `unknown`); `task_spec_type` is `fuzzy` or `specific`; and `dependency_context` is one of `codebase`, `env`, `codebase+env`, or `none`. They describe the evidence needed to interpret an instance and support audit or stratification. They are not additional task categories and do not enter the main expert mapping g_ψ .

Not an implemented label axis. Fault localization, code navigation, test generation, and long-horizon planning are not separately annotated axes in the current label schema.

Frozen structural audit. Table 17 summarizes the final structural snapshot on the 1,531-task collection from SWE-bench Verified, SWE-bench Pro, and SWE-bench Multilingual. Because this collection was used to finalize the inventory, these values measure coverage and internal consistency rather than held-out semantic accuracy.

The audit contains one empty Domain L1 output, and six repositories retain more than one nonempty Domain L1 label. These include genuinely ambiguous products such as a music server that can be viewed as either a web backend or a media system. For reproducibility, the released artifact records the labeling model, active prompt, definition files, raw export, and grouping manifest together; the current JSONL schema does not independently encode all of these artifact identifiers in each record.

Table 17: Structural audit of the frozen SWE Labeler. Repository consistency is computed from nonempty Domain L1 outputs.

Audit item	Final value
Task Type L1 / L2 inventory	26 / 119
Domain L1 / L2 inventory	21 / 108
Orthogonal dimensions / levels	3 / 12
Task-L2 <i>unspecified</i>	70 (4.6%)
Domain-L2 <i>unspecified</i>	35 (2.3%)
Consistent nonempty Domain L1 by repository	58/64 (90.6%)
Persisted illegal concrete L1–L2 assignments	0

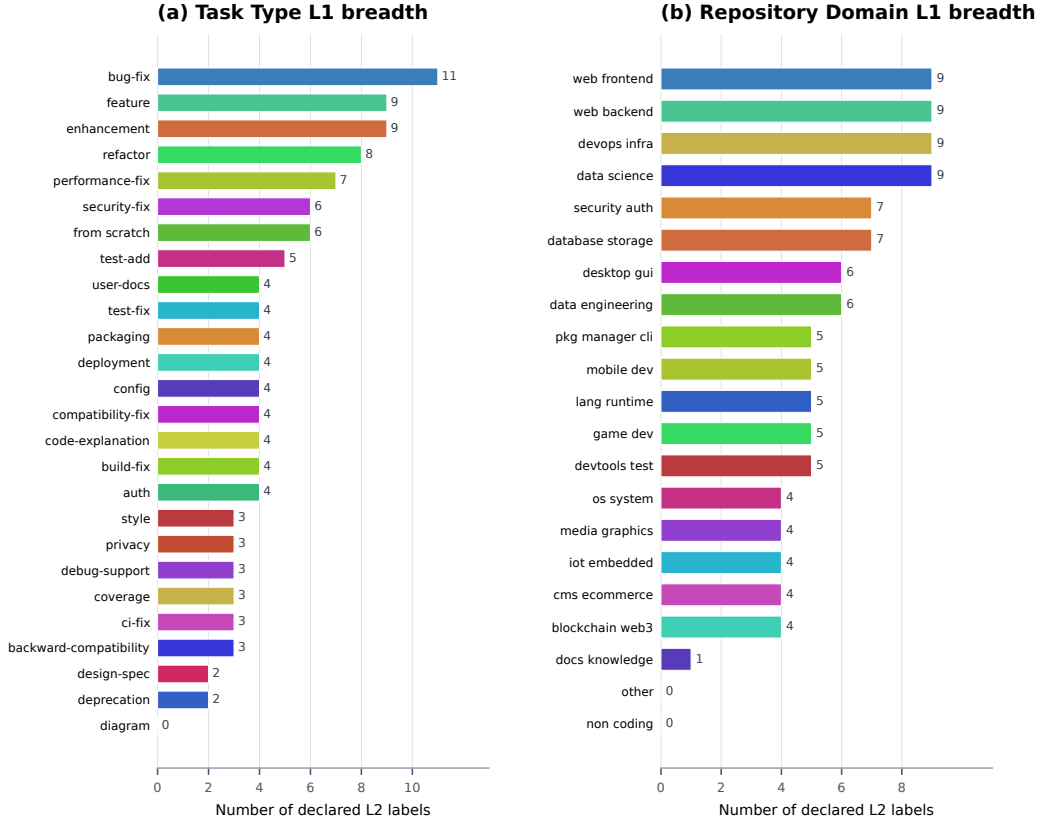


Figure 9: **Exact L2 breadth of every L1 family.** Bars report the number of declared L2 children, sorted within each semantic axis. Zero-child families intentionally fall back to *unspecified*; the values describe taxonomy structure, not empirical sample prevalence.

B.3 Annotation and validation

Instance annotation uses the issue description, repository metadata, gold fix, and test patch as offline evidence. The first phase assigns L1 labels, and the second selects L2 labels from the corresponding L1 families. These annotation inputs are separate from the repair agent’s evaluation input, which does not include the gold fix or test patch.

For full-trajectory annotation, compact evidence additionally retains user requests, code edits, tool actions, and test outcomes. Valid existing L1 labels can be preserved before L2 refinement. Outputs are checked against the declared label inventory and hierarchy: concrete L2 labels must belong to the resolved L1 family, and *unspecified* is used alone. Invalid outputs are excluded from the labeled export.

B.4 Three-route construction and routing specification

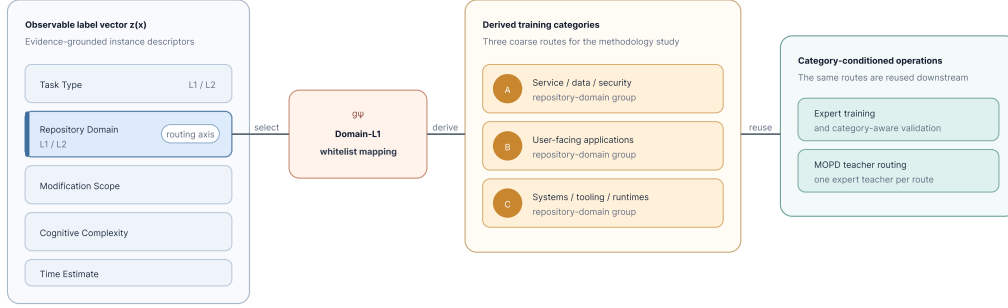


Figure 10: **From SWE labels to category-conditioned training and routing.** Repository Domain L1 is mapped by a fixed whitelist into three derived training categories. The resulting A/B/C assignment is reused for expert training, category-aware validation, and MOPD teacher routing.

The three-route map operates solely on Repository Domain L1. It is deterministic and disjoint: every listed Domain L1 value maps to exactly one route, and all other values map to $\emptyset$. Task Type, L2 labels, and the scale axes are retained for stratification and analysis rather than used as additional routing predicates.

Table 18: Repository Domain L1 mapping for the three-route expert configuration.

Route	Engineering context	Domain L1 whitelist
A	Service/data/security	web_backend, database_storage, data_engineering, data_science, cms_ecommerce, security_auth, blockchain_web3
B	User-facing applications	web_frontend, desktop_gui, mobile_dev, media_graphics, game_dev
C	Systems/tooling/runtimes	devops_infra, devtools_test, pkg_manager_cli, os_system, iot_embedded, lang_runtime

C Training implementation details

C.1 Agentic-miniRL update order

For each Agentic-miniRL batch, the implementation follows this order:

1. Generate multi-turn trajectories with the rollout policy and record inference log probabilities and response-segment masks.
2. Execute the environment and apply RLOO to the response-level returns as in Equation 11.
3. Expand the centered return to the terminal token, subtract the K1 reward, and compute token-level return-to-go as in Equation 14.
4. Recompute current and old trainer log probabilities, form the behavior correction and sign-aware mask in Equation 15, and optimize the complete turn-aware loss in Equation 16.

D Additional training-instance analyses

The training-instance diagnostic aligns the base model, selected Stage-1 checkpoints, and the post-Stage-2 SFT policies on the same matched training records. There are 601, 516, and 1,652 training records in A, B, and C. Each source record contributes equally to the summaries. Repeated records

share their instance-level rate estimates and do not constitute independent probes. The repeated C task has four base attempts and eight post-RL attempts, with a post-RL rate of 5/8.

Each estimate is the observed success count divided by the actual attempt count. In the base data, A and B each contain one eight-attempt instance; C contains nine eight-attempt and three three-attempt instances. All remaining base estimates use four attempts. Post-RL estimates use four attempts except for the duplicated C instance above. Post-SFT estimates use four attempts except for one A instance and seven C instances with eight attempts. The post-SFT refresh probes a broader pool, but only the original matched instances enter this comparison. Failures, including infrastructure failures, remain in the supplied attempt denominators. Category means weight training records equally rather than pooling episodes with different sample counts. Up, same, and down compare the exact success-count fractions before rounding.

Table 19: Training-record success rates before and after initial expert RL. Base and post-RL rates are equal-weight means over matched training records (%); Δ is in percentage points. Up/same/down count records with higher/equal/lower observed pass rates. These are paired empirical estimates, typically from four rollouts per instance.

Expert	N	Base	Post-RL	Δ	Up	Same	Down
A	601	38.79	46.05	+7.26	259	151	191
B	516	38.78	45.54	+6.76	216	154	146
C	1,652	38.68	43.28	+4.60	633	517	502

Repair recovery is evaluated on training records whose initial-RL rates fall below Base and whose membership in the actual SFT data is confirmed. There are 179, 141, and 431 such records for A, B, and C (751 total). Recovery to Base means $p_{\text{SFT}} \geq p_{\text{Base}}$; partial recovery means $p_{\text{RL}} < p_{\text{SFT}} < p_{\text{Base}}$; no improvement means $p_{\text{SFT}} \leq p_{\text{RL}}$. The separate gain-retention cohort requires $p_{\text{RL}} > p_{\text{Base}}$ and $p_{\text{SFT}} < p_{\text{RL}}$ (404 records). Its above/equal/below-Base outcomes use that cohort’s denominator, not all 580 post-SFT decreases. All cohort boundaries use integer-count fractions before rounding.

Table 20: Observed recovery and gain retention after first Repair SFT. Entries give training-record counts with percentages in parentheses. Each percentage uses the category-specific denominator shown in its panel. Regression means an observed initial-RL rate below Base. The second cohort is a subset of all training records whose rates decrease after SFT.

<i>Previously regressed training records included in Repair SFT</i>			
Outcome after Repair SFT	A ($N = 179$)	B ($N = 141$)	C ($N = 431$)
At or above Base	125 (69.8%)	103 (73.0%)	264 (61.3%)
Above RL but below Base	7 (3.9%)	5 (3.5%)	30 (7.0%)
At or below RL	47 (26.3%)	33 (23.4%)	137 (31.8%)
<i>Initial-RL gains followed by a decline after SFT</i>			
Outcome after Repair SFT	A ($N = 103$)	B ($N = 75$)	C ($N = 226$)
Still above Base	36 (35.0%)	28 (37.3%)	71 (31.4%)
Equal to Base	48 (46.6%)	32 (42.7%)	103 (45.6%)
Below Base	19 (18.4%)	15 (20.0%)	52 (23.0%)

To distinguish the magnitudes of gains and losses, let $d_i = p_i^{\text{stage}} - p_i^{\text{Base}}$ for each of the N matched records in a category. The positive contribution is $100N^{-1} \sum_i \max(d_i, 0)$ and the negative contribution is $100N^{-1} \sum_i \min(d_i, 0)$. Their sum equals the stage’s mean gain over Base in percentage points. Both contributions use all matched records as the denominator, rather than separate means over improving and declining subsets. Table 21 reports this decomposition.

These matched snapshots describe observed changes within the selected training set. They are not a longitudinal test of persistent forgetting, and finite rollout sampling contributes to the measured changes. Moreover, the initial pool was selected using base-model mastery estimates, so the diagnostic is conditional on that training-set selection. Selecting instances with low or high observed RL rates also makes regression to the mean relevant to recovery and retention comparisons. These before/after observations do not isolate SFT’s causal effect from a no-SFT control. Actual SFT membership

Table 21: Positive and negative contributions to mean training-record gains relative to Base (percentage points). Each contribution is normalized by all matched training records in that category; positive plus negative equals net gain.

Expert	Stage	Positive	Negative	Net
A	Initial RL	+16.85	−9.59	+7.26
A	Repair SFT	+22.46	−4.97	+17.49
B	Initial RL	+15.12	−8.36	+6.76
B	Repair SFT	+25.61	−4.80	+20.81
C	Initial RL	+13.68	−9.08	+4.60
C	Repair SFT	+20.71	−6.64	+14.07

establishes training-data exposure; a zero-success post-RL probe does not establish an empty historical success buffer.